CAREER OS: AI-POWERED JOB PREPARATION AND CAREER READINESS PLATFORM

A PROJECT REPORT

Submitted by

[STUDENT NAME(S)]

in partial fulfillment for the award of the degree

of

[NAME OF THE DEGREE]

IN

[BRANCH OF STUDY]

[NAME OF THE COLLEGE]

ANNA UNIVERSITY : CHENNAI 600 025

[MONTH & YEAR]

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “CAREER OS: AI-POWERED JOB PREPARATION AND CAREER READINESS PLATFORM” is the bonafide work of “[STUDENT NAME(S)]” who carried out the project work under my supervision.

<<SIGNATURE>>

<<NAME OF THE HEAD OF THE DEPARTMENT>>

HEAD OF THE DEPARTMENT

<<Department>>

<<Full address of the Department and College>>

<<SIGNATURE>>

<<NAME OF THE SUPERVISOR>>

SUPERVISOR

<<Academic Designation>>

<<Department>>

<<Full address of the Department and College>>

ABSTRACT

Students preparing for jobs usually depend on separate tools for resume writing, aptitude practice, interview rehearsal and job search, and none of these tools tells a student how ready they are or what to do next. Career OS is a web-based platform that brings these activities together and gives feedback computed from the student's own saved data. The system uses a Next.js and TypeScript frontend, a FastAPI backend and a relational database accessed through SQLAlchemy. A student can upload a resume that is analysed by documented rule-based formulas, practise from a reviewed bank of 235 questions, train for any job role by choosing or typing a title, receive immediate marks and explanations, and read a complete scored review of every session. A role engine maps any job title to practice topics, prefers questions the student has not seen, adapts difficulty to earlier results and, when a language-model key is configured, generates validated questions to fill gaps. An AI Mentor answers free-form questions through a hosted language model using only verified facts from the student's own records. Recruiters, placement officers and administrators have separate portals. The backend exposes 124 routes and has 483 automated tests, of which 481 passed in the final run; the frontend has 54 passing tests. No model was trained, and persistence on the remote database was not verified.

Keywords: job preparation, resume analysis, adaptive assessment, role-based training, large language model, FastAPI, Next.js

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	ix
1	**INTRODUCTION**	**1**
	1.1 GENERAL AND BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 EXISTING SYSTEM AND ITS LIMITATIONS	2
	1.4 PROPOSED SYSTEM	2
	1.5 OBJECTIVES AND SCOPE	3
	1.6 APPLICATIONS AND EXPECTED OUTCOMES	4
2	**LITERATURE SURVEY AND EXISTING SYSTEM**	**4**
	2.1 JOB PREPARATION AND INTERVIEW PLATFORMS	4
	2.2 RESUME SCREENING SYSTEMS	5
	2.3 ONLINE ASSESSMENT SYSTEMS	5
	2.4 AI-BASED CAREER ASSISTANCE	6
	2.5 LIMITATIONS OF EXISTING APPROACHES AND COMPARISON	6
3	**SYSTEM REQUIREMENTS AND FEASIBILITY STUDY**	**6**
	3.1 FUNCTIONAL REQUIREMENTS	6
	3.2 NON-FUNCTIONAL REQUIREMENTS	7
	3.3 HARDWARE AND SOFTWARE REQUIREMENTS	7
	3.4 TECHNOLOGY AND USER REQUIREMENTS	7
	3.5 FEASIBILITY STUDY	8
4	**SYSTEM ARCHITECTURE AND DESIGN**	**8**
	4.1 OVERALL SYSTEM ARCHITECTURE	8
	4.2 FRONTEND AND BACKEND ARCHITECTURE	9
	4.3 CLIENT-SERVER COMMUNICATION AND API FLOW	11
	4.4 AUTHENTICATION AND AUTHORIZATION FLOW	12
	4.5 AI PROCESSING FLOW	13
	4.6 DATA FLOW DIAGRAMS	14
	4.7 USE CASE AND ACTIVITY DIAGRAMS	16
	4.8 SEQUENCE AND ENTITY RELATIONSHIP DIAGRAMS	17
5	**TECHNOLOGY STACK**	**19**
	5.1 FRONTEND TECHNOLOGIES	20

5.2 BACKEND TECHNOLOGIES 20
5.3 DATABASE, AI AND DATA PROCESSING TECHNOLOGIES 20
5.4 TOOLS, DEPLOYMENT AND DORMANT TECHNOLOGIES 21
6 MODULE DESCRIPTION 21
6.1 USER AUTHENTICATION 21
6.2 STUDENT DASHBOARD AND PROFILE 22
6.3 RESUME MANAGEMENT AND ANALYSIS 22
6.4 JOB RECOMMENDATIONS 22
6.5 INTERVIEW PREPARATION, APTITUDE AND TECHNICAL ASSESSMENTS 23
6.6 ROLE-BASED TRAINING 23
6.7 AI MENTOR 23
6.8 COMMUNICATION PRACTICE AND INTERVIEW STUDIO 24
6.9 CAREER DISCOVERY, ROADMAP AND RECOMMENDATIONS 24
6.10 PLACEMENT, RECRUITER AND ADMIN MANAGEMENT 24
6.11 FEATURE STATUS SUMMARY 25
7 DATABASE DESIGN 25
7.1 DATABASE OVERVIEW AND SELECTION 25
7.2 TABLE GROUPS AND IMPORTANT TABLES 25
7.3 KEYS, RELATIONSHIPS, INDEXES AND INTEGRITY 26
7.4 ORM ACCESS AND USER DATA ISOLATION 27
7.5 DATABASE CONNECTION FLOW 27
7.6 LOCAL TEST DATABASE AND REMOTE DATABASE 28
8 API AND BACKEND IMPLEMENTATION 28
8.1 FASTAPI APPLICATION AND ROUTER STRUCTURE 28
8.2 REQUEST, RESPONSE AND VALIDATION 29
8.3 SERVICE LAYER AND DATABASE SESSIONS 29
8.4 AUTHENTICATION DEPENDENCIES AND AUTHORIZATION 29
8.5 ERROR HANDLING 30
8.6 ENDPOINT INVENTORY 30
9 AI IMPLEMENTATION AND ALGORITHMS 31
9.1 CLASSIFICATION OF THE INTELLIGENT FEATURES 31
9.2 AI MENTOR AND LANGUAGE-MODEL INTEGRATION 32
9.3 RESUME TEXT EXTRACTION AND ANALYSIS 33
9.4 JOB MATCHING, RECOMMENDATIONS AND ASSESSMENT SCORING 34

9.5 ROLE-BASED QUESTION SELECTION AND ADAPTIVE TRAINING 34
9.6 INTERVIEW AND COMMUNICATION EVALUATION, AND LIMITS 36
10 IMPLEMENTATION AND USER INTERFACE 37
10.1 DEVELOPMENT AND RUN ENVIRONMENT 37
10.2 LOGIN AND REGISTRATION 37
10.3 STUDENT DASHBOARD AND RESUME CENTER 37
10.4 INTERVIEW PREPARATION AND ROLE-BASED TRAINING 38
10.5 AI MENTOR AND JOB RECOMMENDATIONS 40
10.6 RECRUITER, PLACEMENT OFFICER AND ADMINISTRATOR INTERFACES 41
11 TESTING AND RESULTS 42
11.1 TESTING STRATEGY AND ENVIRONMENT 42
11.2 UNIT, INTEGRATION AND API TESTING 42
11.3 FRONTEND, DATABASE AND SECURITY TESTING 43
11.4 BROWSER TESTING 43
11.5 SELECTED TEST CASES 44
11.6 TEST SUMMARY AND RESULTS 44
12 RESULTS, LIMITATIONS AND FUTURE ENHANCEMENT 45
12.1 RESULTS AND WORKING FEATURES 45
12.2 KNOWN LIMITATIONS 46
12.3 EXTERNAL DEPENDENCIES AND SECURITY CONSIDERATIONS 47
12.4 FUTURE ENHANCEMENT, SCALABILITY AND DEPLOYMENT 47
13 CONCLUSION 48
APPENDIX 1 SAMPLE API REQUESTS AND RESPONSES 49
REFERENCES 50

LIST OF TABLES

TABLE NO.	TITLE OF THE TABLE	PAGE NO.
1.1	Limitations of the existing approach and the response of Career OS	2
2.1	Qualitative comparison of typical approaches with Career OS	6
3.1	Functional requirements of Career OS	6
3.2	Non-functional requirements and how they are met	7
3.3	Hardware and software requirements	7
5.1	Frontend technologies and their roles	20
5.2	Backend technologies and their roles	20
5.3	Declared but unused or dormant technologies	21
6.1	Module description: User Authentication	21
6.2	Module description: Student Dashboard and Profile	22
6.3	Module description: Resume Management and Analysis	22
6.4	Module description: Job Recommendations	22
6.5	Module description: Interview Preparation, Aptitude and Technical Assessments	23
6.6	Module description: Role-Based Training	23
6.7	Module description: AI Mentor	23
6.8	Module description: Communication Practice and Interview Studio	24
6.9	Module description: Career Discovery, Roadmap and Recommendations	24
6.10	Module description: Placement, Recruiter and Admin Management	24
6.11	Classification of the implemented features	25
7.1	Important tables, keys and columns	26
7.2	Database environments and verification status	28
8.1	Authentication and authorization dependencies	29
8.2	Key API endpoints (paths are relative to /api/v1)	30
9.1	Classification of intelligent features	31
9.2	Marking rules of role-based training	36
11.1	Backend automated tests by layer	42
11.2	Selected test cases, expected and actual results	44
11.3	Summary of verification results (final run, 31 October 2026)	44
12.1	Known limitations	46

LIST OF FIGURES

FIGURE NO. **TITLE OF THE FIGURE** **PAGE NO.**

1.1 Main modules of the Career OS student workspace 3
4.1 Overall architecture of Career OS 9
4.2 Layers of the frontend 10
4.3 Layered architecture of the backend 11
4.4 Request flow between the browser, the API and the database 12
4.5 Authentication and authorization flow 13
4.6 Processing flow of an AI Mentor message 14
4.7 Data flow diagram, level 0 (context diagram) 15
4.8 Data flow diagram, level 1 15
4.9 Use case diagram of Career OS 16
4.10 Activity diagram of a role-based training session 17
4.11 Sequence diagram of answer evaluation 18
4.12 Entity relationship diagram of the core tables 19
7.1 Database connection flow and safety guards 28
9.1 Resume analysis pipeline 33
10.1 Login page (left) and registration page (right) 37
10.2 Student dashboard 38
10.3 Resume Center with the rule-based analysis 38
10.4 Interview Preparation home with progress by round 39
10.5 Role selection with a custom job title 39
10.6 Immediate feedback after submitting an answer 40
10.7 Assessment review: score, counts and topic-wise performance 40
10.8 AI Mentor conversation with suggested next steps 41
10.9 Recommended IT jobs from a live remote-jobs source 41
10.10 Recruiter portal (left) and placement officer portal (right) 42
10.11 Administrator console 42

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

No mathematical symbols are used in this report other than those defined where they appear. Standard abbreviations are listed below.

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity Relationship
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCQ	Multiple-Choice Question
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
STAR	Situation, Task, Action, Result
UI / UX	User Interface / User Experience
UUID	Universally Unique Identifier
WS	WebSocket

CHAPTER 1
INTRODUCTION

1.1 GENERAL AND BACKGROUND

Preparing for a first job is a multi-step task. A student has to write a resume that a screening system can read, practise aptitude and technical questions, learn to speak about projects, rehearse interviews and find openings that match their skills. Most students handle each step with a different website, and the results are never connected: a poor quiz score does not change what the student practises next, and a resume score does not influence the interview preparation. Practice material is plentiful but generic; a student applying for a backend developer position and one applying for a cloud engineer position are usually shown the same question lists. Large studies of hiring practices also report that automated screening can filter out qualified applicants because of rigid rules (Fuller et al., 2021), so students need to understand how machines read their documents.

Career OS is a web-based job-preparation and career-readiness platform developed to close this gap. It keeps the student's profile, resume analysis, practice results, interview reports and goals in one database and uses them to produce feedback and recommendations for four kinds of users: students, recruiters, placement officers and administrators. This report describes the design (Chapters 3 and 4), technology and modules (Chapters 5 and 6), database and backend interface (Chapters 7 and 8), algorithms (Chapter 9), screens (Chapter 10), testing (Chapter 11), results and limitations (Chapter 12) of the system as it exists in the project repository.

1.2 PROBLEM STATEMENT

Students lack a single place that (i) measures how complete and readable their resume is, (ii) lets them practise questions that match the job role they want, however unusual the title, (iii) gives immediate and explained feedback on every answer, (iv) remembers what they have already practised so that new

sessions contain fresh questions, and (v) answers their free-form questions without inventing facts about them. The problem addressed by Career OS is the design and implementation of such a platform with honest, traceable scoring.

1.3 EXISTING SYSTEM AND ITS LIMITATIONS

The usual approach is a collection of independent tools: a word-processor template for the resume, free online quizzes for aptitude, video tutorials for interviews, a job portal for searching and a general chatbot for questions. Each has its own login, its own data and its own idea of progress. Table 1.1 summarises the limitations and the response of Career OS.

Table 1.1 Limitations of the existing approach and the response of Career OS

Limitation	Response in Career OS
Tools are disconnected, so there is no overall view of readiness	One database; recommendations read resume, practice and goals together
Practice is not role-specific and questions repeat	Role engine maps any job title to topics; a seen-question history prefers unseen questions
Feedback is delayed, vague or unexplained	Immediate result, correct answer, explanation and suggestion, labelled by scoring method
General chatbots may invent facts about the user	The mentor receives only verified records and refuses to chat without a configured provider

1.4 PROPOSED SYSTEM

Career OS is proposed as an integrated platform with a web frontend, an application server and a relational database. After registration a student maintains a profile, uploads a resume, practises by round or by job role, rehearses interviews, chats with an AI Mentor and follows a learning roadmap (Figure 1.1). Recruiters publish jobs and review applications, placement officers organise campus drives and rank students, and administrators manage users, roles and announcements.

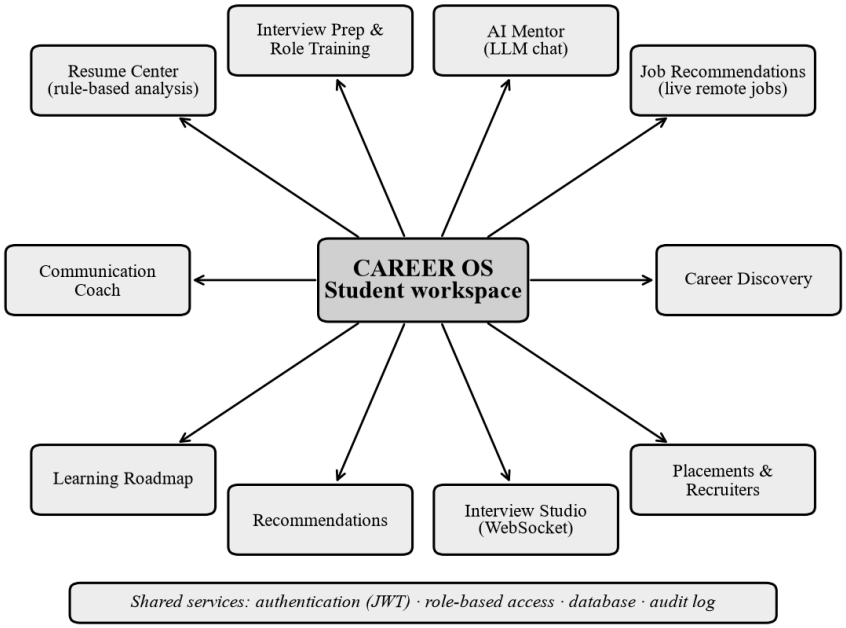


Figure 1.1 Main modules of the Career OS student workspace

The system is deliberately honest about its intelligence. Resume analysis, interview and communication feedback, recommendations, the roadmap and placement ranking are rule-based algorithms with documented formulas. A large language model is used in one place only, the AI Mentor, and optionally to generate and grade role-specific questions when a provider key is configured. No machine-learning model is trained in this project.

1.5 OBJECTIVES AND SCOPE

1. Provide secure registration and login with role-based access for four user groups.
2. Analyse an uploaded resume and report completeness, readability and wording with transparent formulas.
3. Let a student practise for any job role, prefer unseen questions, adapt difficulty and explain the choice.
4. Evaluate every answer immediately and compute final scores on the server from saved records.
5. Provide an AI Mentor that uses only verified facts from the student's account, and portals for recruiters, officers and administrators.

6. Verify the implementation with automated tests and report its limitations honestly.

The scope covers the web application, the backend services, the database design and the automated tests. It excludes a mobile application, payments, e-mail delivery, social sign-in, speech or video analysis and execution of student-written code. Question content is a reviewed bank of 235 questions extended by optional AI generation. The target users are students (preparation, resume feedback, mentor), recruiters (jobs and applications), placement officers (drives and ranking) and administrators (users, roles, audit). Scores are practice feedback, not predictions of hiring decisions.

1.6 APPLICATIONS AND EXPECTED OUTCOMES

A college placement cell can prepare students before campus drives and rank registered students; a training institute can use the role-based engine for roles that standard question sets do not cover; an individual job seeker can use the resume analysis and the mentor to find missing skills for a target role. The expected outcome is a working, tested platform in which a student can complete the full cycle of analysing a resume, practising for a role, seeing immediate feedback, reading a scored review and following suggested next steps, with every figure traceable to saved data.

CHAPTER 2
LITERATURE SURVEY AND EXISTING SYSTEM

This chapter reviews the approaches that students and institutions use today and the ideas on which Career OS builds. Product categories are described in general terms and no vendor statistics are quoted; scholarly sources are cited only where their content was checked, and all are listed in the References.

2.1 JOB PREPARATION AND INTERVIEW PLATFORMS

Preparation material is delivered through job portals that list vacancies, assessment sites that offer timed quizzes and coding problems, interview-

practice services that connect candidates with peers or mentors, and learning platforms that offer courses. Job portals concentrate on search and applications, and assessment sites on a limited set of skills. Interview services typically offer question lists by company or topic, recorded mock interviews or live sessions with people. Question lists are usually static, so a student who practises regularly sees the same questions again. In general, none of these channels combines resume feedback, role-specific practice and a conversational assistant under one account.

2.2 RESUME SCREENING SYSTEMS

Employers use applicant tracking systems to store and filter applications. Fuller et al. (2021) studied hiring practices in several countries and concluded that automated systems that rely on rigid exclusion rules can screen out candidates who could do the job. For students this means that a resume must be machine-readable and must state skills clearly. Keyword matching is the basic technique behind many such checks; it is related to term weighting in information retrieval (Salton and Buckley, 1988) and to set-overlap measures such as the Jaccard index (Jaccard, 1912). Career OS uses transparent keyword and structure measurements and states clearly that its scores are not those of any employer's system.

2.3 ONLINE ASSESSMENT SYSTEMS

Educational research supports frequent, low-stakes testing with feedback. Roediger and Karpicke (2006) showed that taking a memory test improved long-term retention compared with repeated study, and Black and Wiliam (1998) reviewed evidence that formative assessment raises learning achievement. These findings motivate the design of Career OS, where every answer is graded immediately and explained and weak topics are revisited in later sessions. Multiple-choice questions can be graded exactly, whereas free-text answers need a rubric; Career OS uses an expected-concept rubric and, when available, a language model to judge semantic equivalence.

2.4 AI-BASED CAREER ASSISTANCE

Large language models are based on the Transformer architecture (Vaswani et al., 2017). Brown et al. (2020) showed that a large model can perform new tasks from a few examples placed in the prompt, which makes it possible to adapt behaviour by prompt design instead of training. Wei et al. (2022) described chain-of-thought prompting for reasoning tasks, and Lewis et al. (2020) introduced retrieval-augmented generation, in which a model is grounded in retrieved text. These ideas explain why Career OS supplies verified facts from the student's own records to the mentor instead of relying on the model's memory. Such models can produce fluent but wrong statements, so the platform restricts the context, caps the output and warns the student.

2.5 LIMITATIONS OF EXISTING APPROACHES AND COMPARISON

Separate tools do not share data, question sets are static or role-agnostic, general chatbots may state invented details with equal confidence, and scores are often unexplained. Table 2.1 compares typical approaches qualitatively with Career OS.

Table 2.1 Qualitative comparison of typical approaches with Career OS

Capability	Job portals	Quiz sites	General chatbot	Career OS
Resume feedback	Limited	Rare	Possible, unverified	Rule-based analysis of the file
Practice for any job title	No	Fixed categories	Possible, unvalidated	Role engine, validated questions
Fresh-question history	Not applicable	Rarely	No memory of practice	Per-student seen map
Explained instant feedback	No	Often	Yes, may be wrong	Yes, labelled by method
Uses the student's own records	Profile only	Scores only	Only if pasted	Verified facts in the mentor

CHAPTER 3

SYSTEM REQUIREMENTS AND FEASIBILITY STUDY

3.1 FUNCTIONAL REQUIREMENTS

Table 3.1 Functional requirements of Career OS

ID	Requirement	Status
FR1	Register, log in, refresh a session; maintain a profile and target role	Implemented, tested

ID	Requirement	Status
FR2	Upload a PDF or DOCX resume, analyse it and download the stored file	Implemented, rule-based
FR3	Practise aptitude, technical, coding, communication, HR and managerial rounds	Implemented, tested
FR4	Practise for any job role with fresh questions and a scored review	Implemented, tested
FR5	Run live interview and communication sessions over WebSocket	Implemented, rule-based
FR6	Chat with an AI Mentor and keep the conversation history	Implemented; needs an LLM key
FR7	Suggest next steps and a learning roadmap; list recommended remote jobs	Implemented; jobs need internet
FR8	Recruiters: companies, jobs, applications. Officers: colleges, drives, eligibility, ranking	Implemented
FR9	Administration, community, dashboards and notifications	Implemented (partly basic)

3.2 NON-FUNCTIONAL REQUIREMENTS

Table 3.2 Non-functional requirements and how they are met

Quality	How it is met
Security and privacy	bcrypt hashes, JWT, role and ownership checks, login throttling, upload validation; every personal resource is filtered by the authenticated user
Reliability	Typed provider errors; nothing is saved when an external service fails
Performance	Asynchronous server, React Query caching, bounded conversation history
Maintainability	Router, service, repository and model layers; 124 documented routes
Honesty and portability	Scores labelled by method; runs on SQLite locally and on PostgreSQL

3.3 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.3 Hardware and software requirements

Item	Requirement
Hardware (development)	Dual-core 2 GHz processor, 8 GB RAM, 5 GB free storage, internet access for the LLM and jobs services
Hardware (small deployment, estimated)	2 virtual CPUs, 4 GB RAM, 20 GB storage, public HTTPS endpoint
Operating system	Windows 11 (development); Linux container (deployment)
Languages and runtimes	Python 3.12 in the Dockerfile (3.14 on the development machine); Node.js 24; TypeScript 5
Frameworks	FastAPI with Uvicorn; Next.js 15.5 with React 19.1
Database	PostgreSQL (Supabase) or SQLite
Tools and browser	Git, npm, pip, pytest, Vitest, ESLint, ruff; any current Chromium-based browser or Firefox

3.4 TECHNOLOGY AND USER REQUIREMENTS

The project requires an HTTPS-capable application server, a PostgreSQL-compatible database for production, an optional LLM account (Groq, xAI or Anthropic) for the mentor and generation features, and outbound access to the public Himalayas jobs interface. Secrets such as the signing key, database URL

and provider key are supplied only through environment variables and never sent to the browser. Students need clear feedback, fresh practice and a mentor that does not invent facts; recruiters need to publish jobs and move applicants through stages; placement officers need eligibility and ranking tools limited to their own college; administrators need role management and an audit trail.

3.5 FEASIBILITY STUDY

Technical feasibility. Every component uses mature open-source technology. The backend and frontend were built, tested and run locally, the production build of the frontend succeeds, and a real language-model request through the configured provider was verified. The only external dependencies are the optional provider and the public jobs interface, both with defined failure behaviour.

Operational feasibility. Students and staff use the system through a web browser with no installation, and the four user groups have separate portals. Limitations, such as the absence of e-mail verification, are stated in the interface where relevant.

Economic feasibility. All frameworks are free software. The recurring costs are hosting and, optionally, LLM usage billed by the provider. The platform keeps these costs low by using stored questions first and calling the LLM only to fill gaps, with per-user limits on messages and generated questions. A cost estimate for a particular deployment was not measured.

CHAPTER 4
SYSTEM ARCHITECTURE AND DESIGN

4.1 OVERALL SYSTEM ARCHITECTURE

Career OS follows a three-tier client-server architecture (Figure 4.1). The client tier is a Next.js application running in the browser. The application tier is a FastAPI server that validates requests, enforces authentication and authorization, applies business rules and talks to the data tier: a relational

database accessed through SQLAlchemy, local file storage for resumes and two optional external services, a language-model provider and a public remote-jobs interface. The browser never talks to the database, and the provider key is held only by the server.

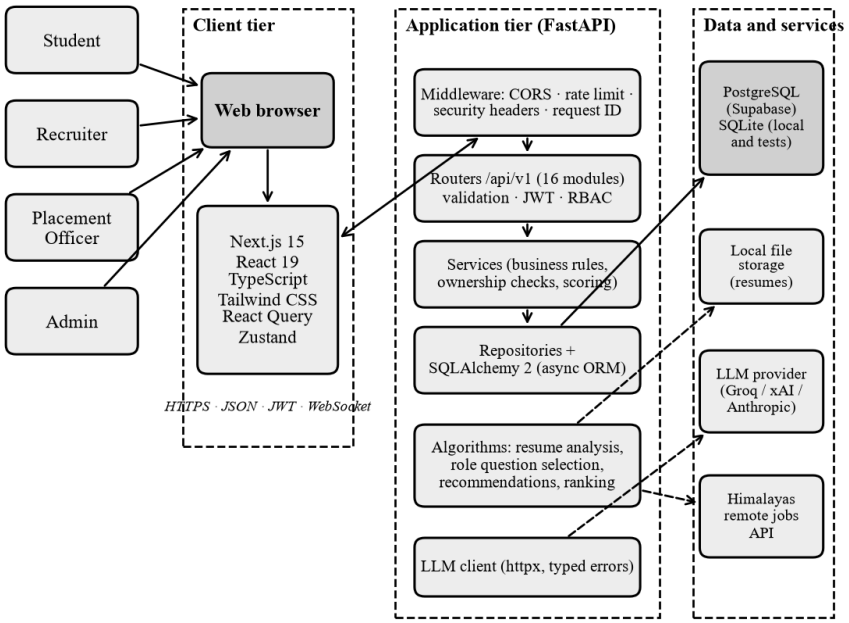


Figure 4.1 Overall architecture of Career OS

4.2 FRONTEND AND BACKEND ARCHITECTURE

The frontend (Figure 4.2) is organised in layers. Pages under the Next.js App Router are thin: the login and registration pages live in the `(auth)` group and all signed-in pages in the `(dashboard)` group. Each page imports a feature module, such as the resume centre or the AI Mentor. Feature modules use shared interface components and two kinds of state: React Query caches server data, and three Zustand stores keep the session, the mentor conversation and notifications. All network access goes through one typed client, `lib/api.ts`, which attaches the access token, refreshes it once when the server answers 401 and converts every failure into an `ApiError` with a readable message.

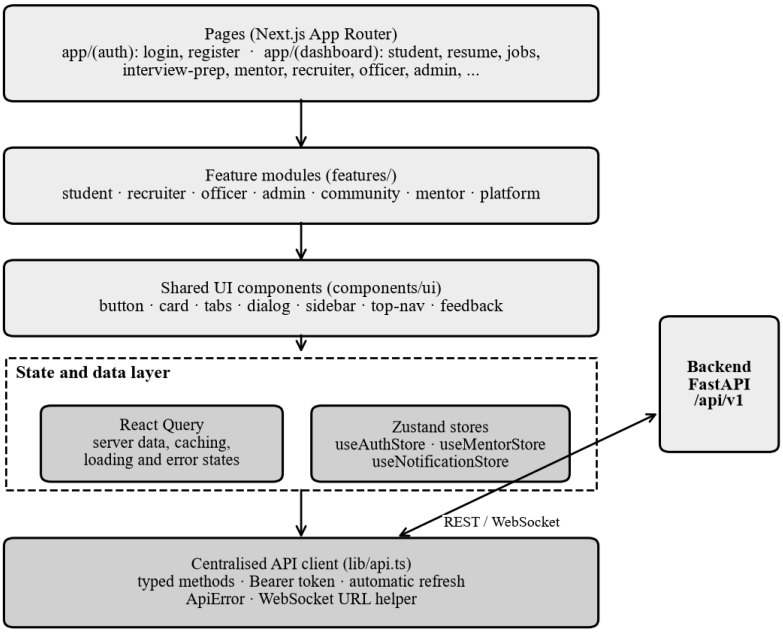


Figure 4.2 Layers of the frontend

The backend (Figure 4.3) processes each request through middleware (request identifier, security headers, rate limiter and cross-origin handling), a router that validates the body with Pydantic and resolves authentication dependencies, a service that applies business rules and ownership checks, and a repository that runs SQLAlchemy queries through an asynchronous session. Shared components such as configuration, password hashing, login throttling, text analysis and the language-model client sit beside this pipeline.

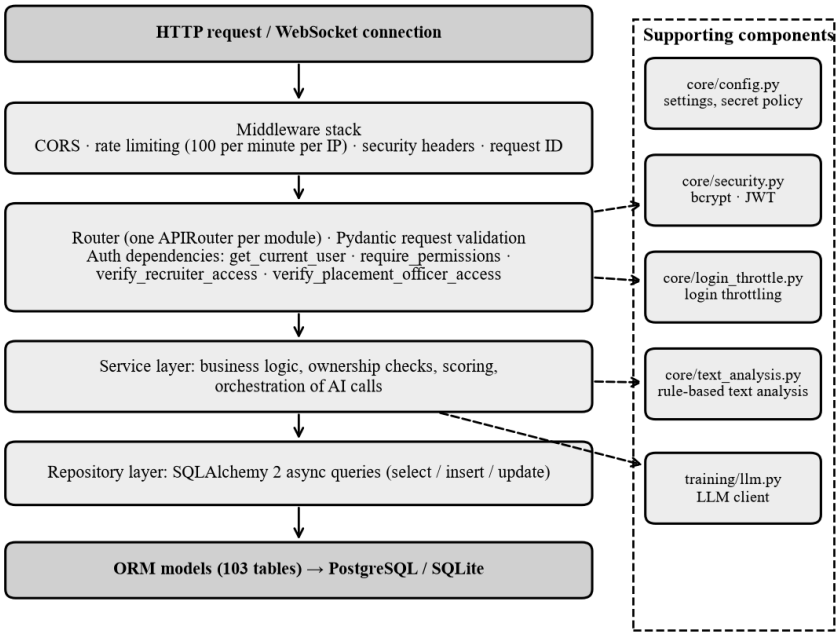


Figure 4.3 Layered architecture of the backend

4.3 CLIENT-SERVER COMMUNICATION AND API FLOW

The frontend and backend exchange JSON over HTTPS using REST conventions (Fielding, 2000). Live interview and communication sessions use WebSocket connections (Fette and Melnikov, 2011) that carry the access token in the query string, because browsers cannot add headers to a WebSocket handshake. Figure 4.4 traces one ordinary request from a user action to the interface update.

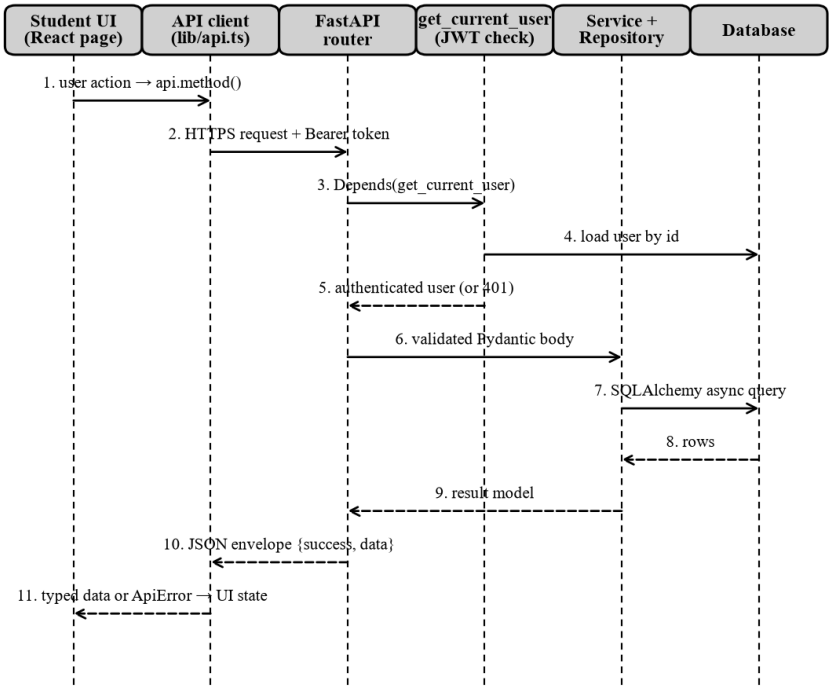


Figure 4.4 Request flow between the browser, the API and the database

4.4 AUTHENTICATION AND AUTHORIZATION FLOW

Authentication and authorization are separate steps (Figure 4.5). During login the server checks the login throttle (five failures per account and thirty per address in fifteen minutes), verifies the password hash with bcrypt (Provos and Mazières, 1999) and issues a short-lived access token and a longer-lived refresh token, both JSON Web Tokens (Jones et al., 2015). On every later request the server checks the signature, expiry and type of the token and that the user is active. Authorization then depends on the route: personal data is protected by an ownership check, administrator routes need named permissions, and recruiter and officer routes need membership of a company or a college.

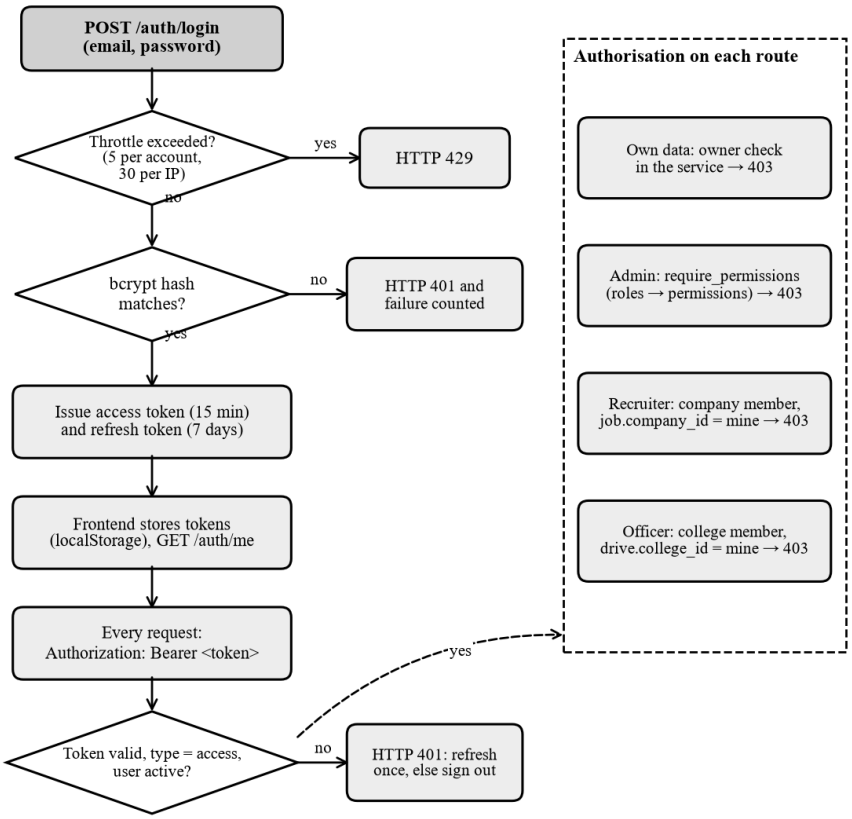


Figure 4.5 Authentication and authorization flow

4.5 AI PROCESSING FLOW

Figure 4.6 shows how a message to the AI Mentor is processed. The server verifies the user and the ownership of the conversation, applies a limit of twelve messages per minute and refuses to continue when no provider key is configured, so that it never produces a pretend answer. Otherwise a context aggregator reads the student's own saved records, the last ten messages are added as history and one request is sent to the provider. Only a successful reply is saved; every failure is converted to a safe message.

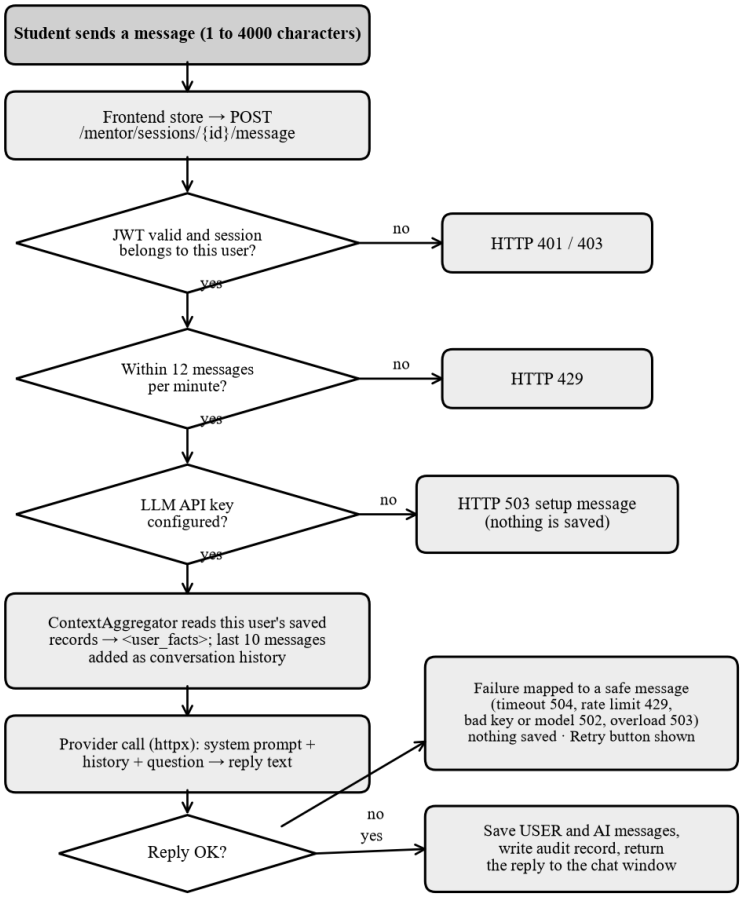


Figure 4.6 Processing flow of an AI Mentor message

4.6 DATA FLOW DIAGRAMS

The context diagram (Figure 4.7) treats the platform as one process: the four user groups send data in and receive scores, feedback, reports and rankings, while the language-model provider and the jobs interface are external systems that the platform queries. Figure 4.8 expands the system into seven processes and seven data stores; each process reads and writes only its own tables.

Arrows show data flowing into and out of the system; results returned to users are scores, feedback, reports and rankings

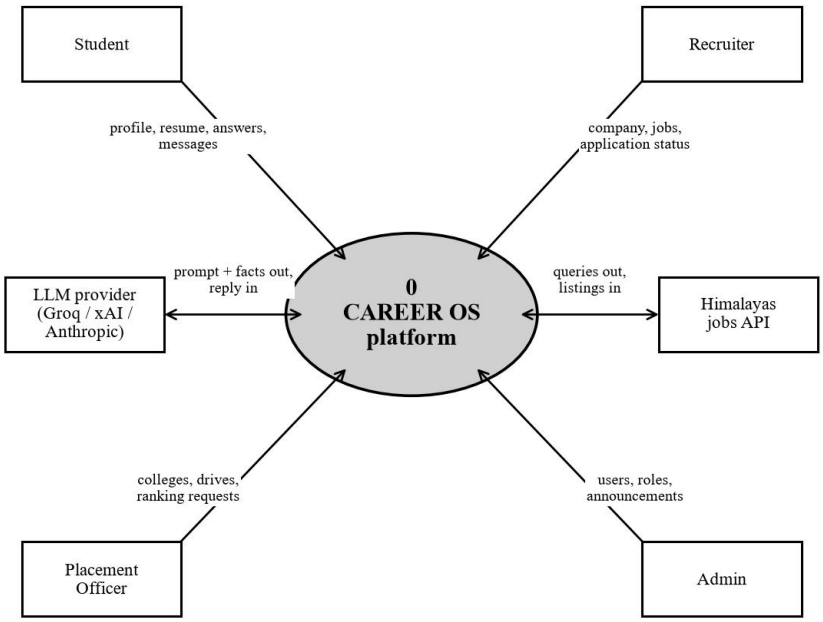
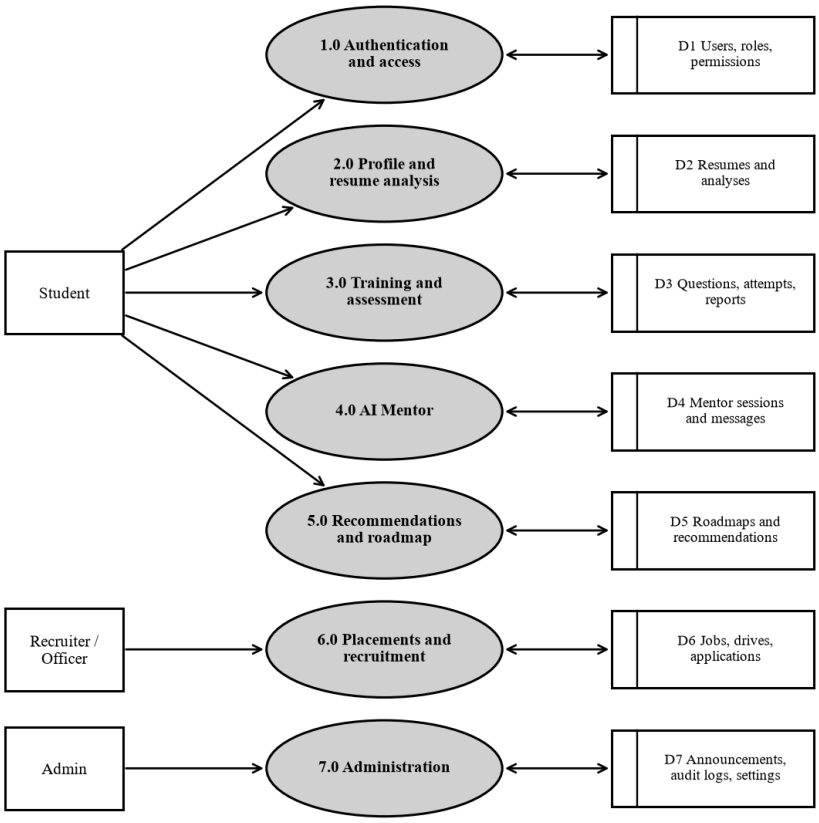


Figure 4.7 Data flow diagram, level 0 (context diagram)

Every process except 1.0 first passes through 1.0 (JWT check); those arrows are omitted for readability.



Data stores are database tables (SQLAlchemy models); resume files are kept in local file storage.

Figure 4.8 Data flow diagram, level 1

4.7 USE CASE AND ACTIVITY DIAGRAMS

Figure 4.9 lists the main use cases of each actor; the student has the largest set because the platform is centred on preparation. The activity diagram (Figure 4.10) follows one role-based training session from the choice of a job role to the final review. Its branches show the honest handling of limited question banks: unseen stored questions are used first, a validated AI call fills only the gap, and repeats are marked when no alternative exists.

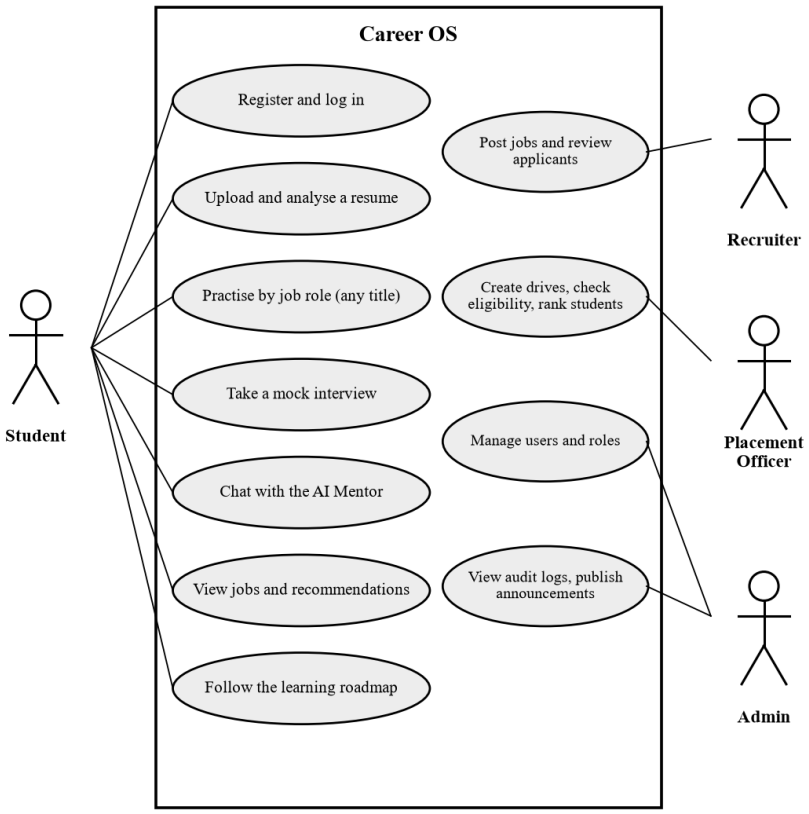


Figure 4.9 Use case diagram of Career OS

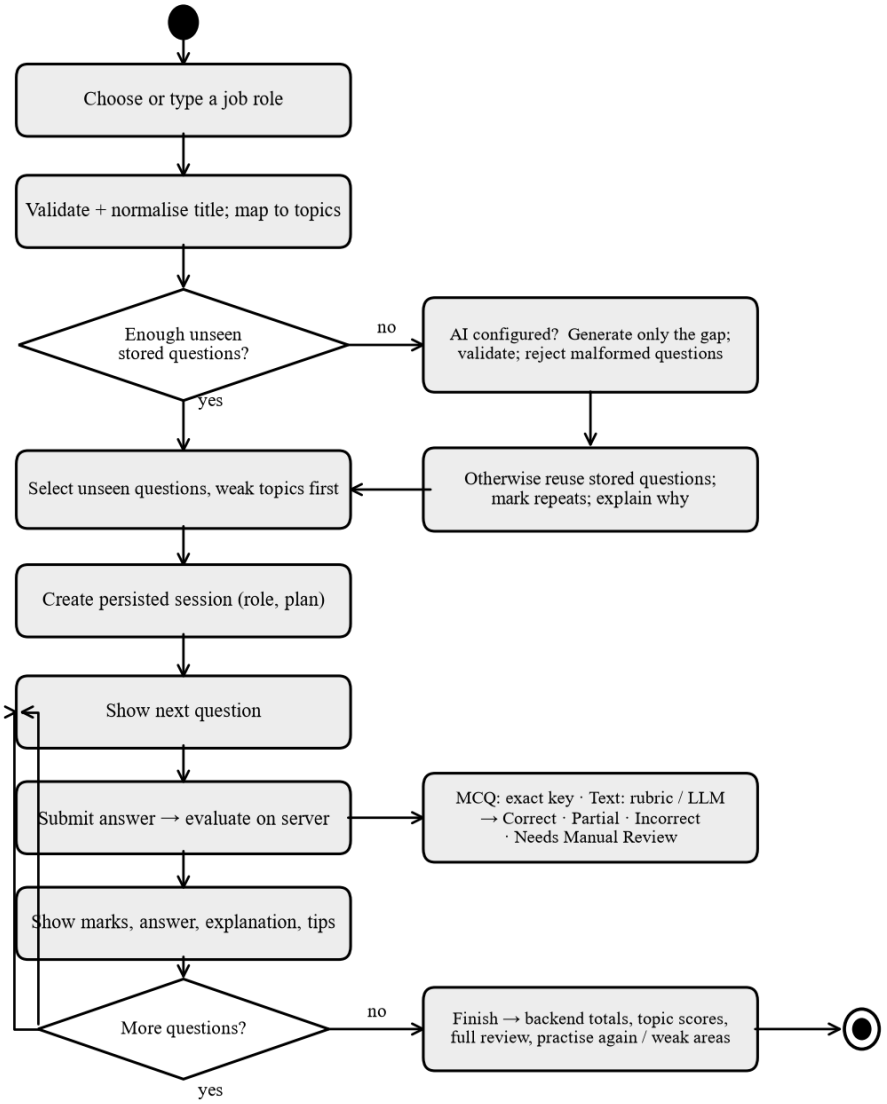


Figure 4.10 Activity diagram of a role-based training session

4.8 SEQUENCE AND ENTITY RELATIONSHIP DIAGRAMS

Figure 4.11 shows how one submitted answer is evaluated. The service loads the session of the signed-in student only, grades a multiple-choice answer by exact comparison or scores a written answer against the expected concepts. If a provider key is configured, a language model may grade the answer; its JSON output is validated and replaced by the rule-based result if invalid. The database holds 103 tables; Figure 4.12 shows the entities central to the student journey, and Chapter 7 explains the design.

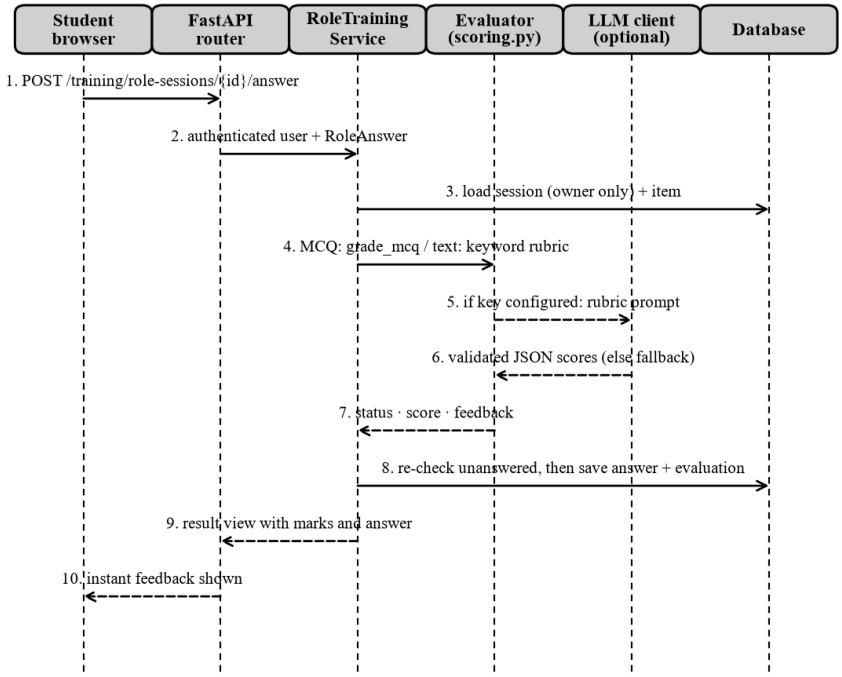


Figure 4.11 Sequence diagram of answer evaluation

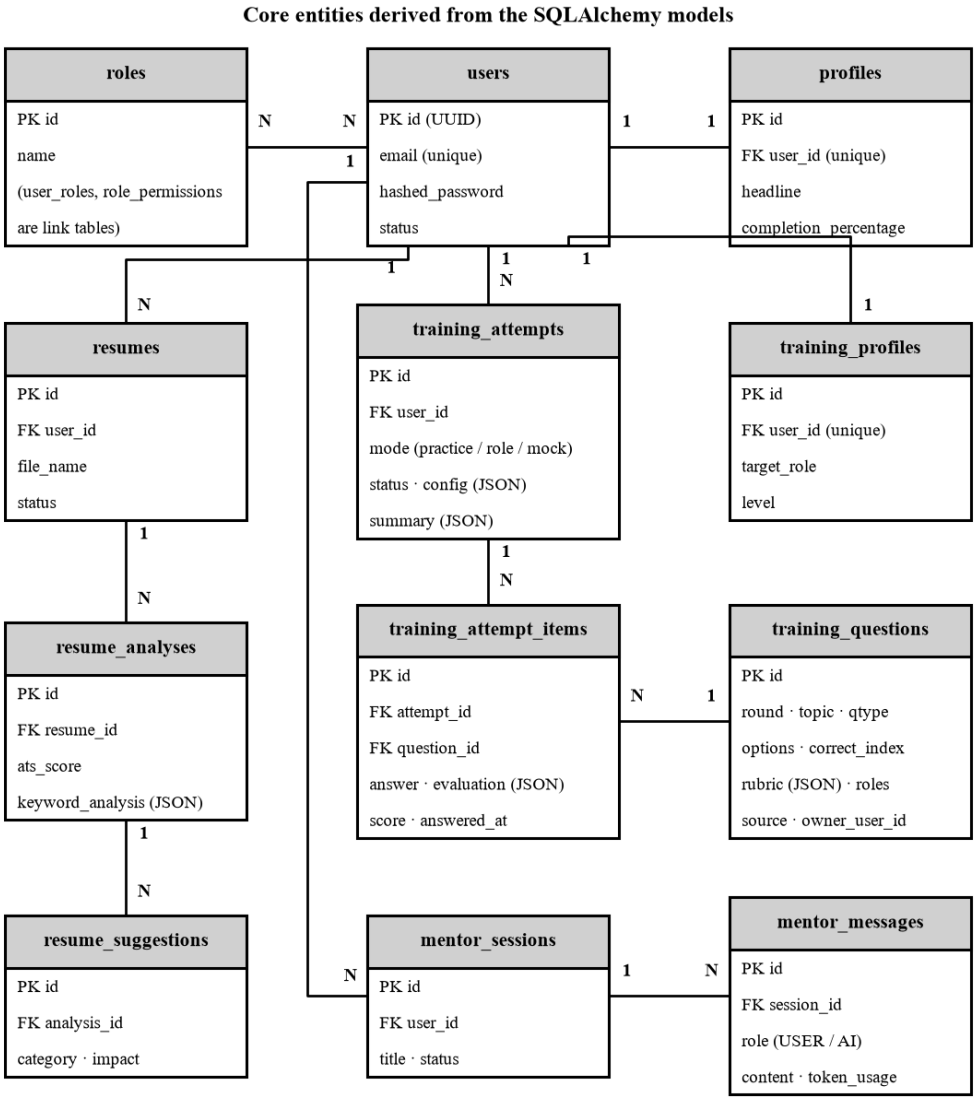


Figure 4.12 Entity relationship diagram of the core tables

CHAPTER 5
TECHNOLOGY STACK

Technologies were selected for strong documentation, support for asynchronous input and output, and a type system that catches interface mismatches early. Only technologies that the project imports or executes are described as used; packages that are declared but dormant are listed at the end of this chapter.

5.1 FRONTEND TECHNOLOGIES

Table 5.1 Frontend technologies and their roles

Technology	What it is	Why selected	Where used in Career OS
Next.js 15.5 and React 19.1	React framework with file-based routing; UI library	Routing, layouts, production build; reusable components	All pages in app/(auth) and app/(dashboard)
TypeScript 5	JavaScript with static types	Compile-time checks of API shapes	All source files; type check passes with zero errors
Tailwind CSS 4	Utility-first styling	Fast, consistent styling with dark mode	All components
React Query 5	Server-state library	Caching, loading and error states	More than 25 files (dashboard, resume, jobs, role training)
Zustand 5	Small client-state store	Global state without boilerplate	Authentication, mentor chat, notifications
react-hook-form, zod	Form handling and validation	Typed form validation	Login and registration forms
Vitest, Testing Library	Test runner and component testing	Fast tests in a simulated browser	54 frontend tests

5.2 BACKEND TECHNOLOGIES

Table 5.2 Backend technologies and their roles

Technology	What it is	Why selected	Where used in Career OS
Python	General-purpose language	Rich libraries for text and data work	All backend code (about 18,400 lines)
FastAPI and Uvicorn	Asynchronous web framework and server	Automatic validation, OpenAPI docs, WebSockets	app/main.py and 16 routers
Pydantic 2	Data validation and typed settings	Rejects bad input before handlers run	Schemas of every module; core/config.py
SQLAlchemy 2 (async)	Object-relational mapper	Portable queries for PostgreSQL and SQLite	models.py and repository.py of each module
asyncpg, aiosqlite	Database drivers	Asynchronous access to PostgreSQL and SQLite	Production and local or test databases
python-jose, bcrypt	JWT library; password hashing	Stateless sessions; slow salted hashes	core/security.py
httpx, pypdf	Async HTTP client; PDF text reader	Provider and jobs calls; offline resume parsing	training/llm.py, jobs/service.py, resumes/text_extraction.py
pytest, pytest-asyncio, ruff	Test framework and linter	Asynchronous tests; consistent style	483 backend tests

5.3 DATABASE, AI AND DATA PROCESSING TECHNOLOGIES

PostgreSQL, provided by Supabase, is the intended production database. The application reads the connection string from the `DATABASE_URL` environment variable, rewrites it for the asyncpg driver and disables the prepared-statement cache so that it works behind a connection pooler. For development and tests the same models run on SQLite. Tables are defined with SQLAlchemy models; there is no migration tool (see Chapter 12).

- **Language-model access:** a provider-neutral client sends chat-completion requests over HTTPS to Groq (model `openai/gpt-oss-120b`), xAI or Anthropic; the key is read only from the server environment.
- **Text processing:** pypdf and a zip-and-XML reader extract PDF and DOCX text locally; a shared module matches a skills dictionary and counts verbs, sections, contact details and slips.
- **External job data:** the public Himalayas remote-jobs interface supplies live listings that are scored against the student's skills.

5.4 TOOLS, DEPLOYMENT AND DORMANT TECHNOLOGIES

Source code is managed with Git and hosted on GitHub; packages are installed with npm and pip; configuration is read from environment variables and the real `.env` file is excluded from version control. Dockerfiles, compose files and Kubernetes manifests exist, but they were not built or run during this work, and no continuous-integration workflow is configured. Table 5.3 lists packages that are declared but are not part of the verified system.

Table 5.3 Declared but unused or dormant technologies

Technology	Status in the repository
Alembic	Listed in requirements; no migration directory exists
passlib, prometheus-client, gunicorn	Listed; not used (bcrypt is called directly; /metrics returns fixed text)
Celery and Redis	Task and cache code exists but no worker or Redis server was run; dormant
S3 storage, e-mail, OAuth	Interfaces or stubs only; resumes are stored on local disk

CHAPTER 6
MODULE DESCRIPTION

Each module is described by its purpose, input, processing, output, frontend and backend implementation, database interaction and current limitations. File names refer to the project repository.

6.1 USER AUTHENTICATION

Table 6.1 Module description: User Authentication

Aspect	Description

Aspect	Description
Purpose	Create accounts and control who can use which feature.
Input	E-mail and password (at least eight characters for new accounts); refresh token.
Processing	Throttle check, bcrypt verification, token issue and refresh, identity and permission lookup.
Output	Access and refresh tokens; the current user with roles, permissions and company or college link.
Frontend	(auth) pages, store/useAuthStore.ts, lib/api.ts token handling.
Backend	auth/router.py, service.py, core/security.py, dependencies.py, login_throttle.py.
Database	users, roles, permissions, user_roles, role_permissions.
Limitations	No e-mail verification or password reset; tokens in browser storage cannot be revoked; the throttle is in memory per process.

6.2 STUDENT DASHBOARD AND PROFILE

Table 6.2 Module description: Student Dashboard and Profile

Aspect	Description
Purpose	Show readiness at a glance and keep the student's profile.
Input	Profile fields and saved results from other modules.
Processing	Aggregates the latest resume score, interview score and roadmap progress; readiness is the mean of the components that have data.
Output	Readiness ring, score cards, quick actions and tasks.
Frontend	app/(dashboard)/student and profile pages (React Query).
Backend	users and analytics routers; ContextAggregator.
Database	profiles, educations, projects, profile_skills, analytics_events.
Limitations	Readiness is a transparent average, not a prediction; some analytics screens are basic.

6.3 RESUME MANAGEMENT AND ANALYSIS

Table 6.3 Module description: Resume Management and Analysis

Aspect	Description
Purpose	Tell the student how complete, readable and well worded the resume is.
Input	A PDF or DOCX file of at most 5 MB and the target role.
Processing	Validate type, size and file signature; store; extract text; compute the scores of Section 9.3 in a background task.
Output	ATS readiness, quality and writing scores, found and missing skills, suggestions; the stored file on request.
Frontend	features/student/resume-center.tsx.
Backend	resumes/upload_validation.py, text_extraction.py, analysis.py, service.py.
Database	resumes, resume_analyses, resume_suggestions; file in local storage.
Limitations	Rule-based, not an LLM judgement; scanned PDFs are flagged, not read; scores do not match any employer system.

6.4 JOB RECOMMENDATIONS

Table 6.4 Module description: Job Recommendations

Aspect	Description
Purpose	Show current remote IT jobs matched to the student's skills.
Input	Keyword, role, experience and location filters.
Processing	Calls the public Himalayas interface, cleans descriptions and scores skill overlap with saved and extracted skills.

Aspect	Description
Output	Paged job cards with a skill-match estimate and a link to the listing.
Frontend	features/student/recommended-jobs.tsx.
Backend	jobs/service.py, skills.py, router.py.
Database	Reads profile skills; listings are not stored.
Limitations	Needs internet; remote roles only; data belongs to a third party.

6.5 INTERVIEW PREPARATION, APTITUDE AND TECHNICAL ASSESSMENTS

Table 6.5 Module description: Interview Preparation, Aptitude and Technical Assessments

Aspect	Description
Purpose	Practise by round and topic with immediate scoring and progress tracking.
Input	Round, topic, difficulty, number of questions, mode (practice, exam, mock).
Processing	Selects questions with a seen map and topic weights; grades multiple-choice answers exactly; scores written answers with a rubric or optional LLM; computes progress from saved answers.
Output	Question sets, feedback, summaries and a roadmap of next steps.
Frontend	features/student/interview-prep (home, round practice, runner, mock).
Backend	training/service.py, scoring.py, progress.py, bank/ (235 questions).
Database	training_questions, training_attempts, training_attempt_items, training_profiles.
Limitations	Coding answers are self-assessed because code is never executed; written scores are estimates.

6.6 ROLE-BASED TRAINING

Table 6.6 Module description: Role-Based Training

Aspect	Description
Purpose	Let a student prepare for any job title with fresh questions, instant marks and a full review.
Input	A chosen or typed job title, level, difficulty, topics, question count and focus.
Processing	Normalises the title, maps it to topics, selects unseen questions, optionally generates validated questions, evaluates each answer and computes totals from saved evaluations.
Output	A session with a plan, per-answer status and marks, and a final review with a practise-again option.
Frontend	interview-prep/role-training.tsx.
Backend	training/roles.py, role_training.py, role_generation.py; nine routes.
Database	training_attempts (mode role), training_attempt_items, training_questions.
Limitations	Reviewed questions exist for 22 catalogued roles; other titles need an LLM key; generated answer keys are validated, not guaranteed correct.

6.7 AI MENTOR

Table 6.7 Module description: AI Mentor

Aspect	Description
Purpose	Answer free-form career and learning questions in English.
Input	A message of 1 to 4000 characters.
Processing	Checks ownership and rate limit, reads verified facts, builds the system prompt and history, calls the provider and saves a successful reply.
Output	An answer plus separate rule-based suggested next steps.
Frontend	features/mentor, features/student/mentor.tsx, store/useMentorStore.ts.

Aspect	Description
Backend	mentor/service.py, prompt.py, router.py; training/llm.py.
Database	mentor_sessions, mentor_messages.
Limitations	Needs a provider key and internet; answers can be wrong; replies are shown as plain text.

6.8 COMMUNICATION PRACTICE AND INTERVIEW STUDIO

Table 6.8 Module description: Communication Practice and Interview Studio

Aspect	Description
Purpose	Practise written communication and live mock interviews.
Input	Typed messages or answers sent over a WebSocket.
Processing	Counts filler, hedging and informal words and slips; scores interview answers with the training rubric.
Output	Live feedback and a report.
Frontend	student/communication.tsx, interview.tsx, lib/useSessionSocket.ts.
Backend	communication and interviews modules (WebSocket routes).
Database	communication_* and interview_* tables.
Limitations	Text only; no speech or video analysis.

6.9 CAREER DISCOVERY, ROADMAP AND RECOMMENDATIONS

Table 6.9 Module description: Career Discovery, Roadmap and Recommendations

Aspect	Description
Purpose	Suggest suitable careers, a learning path and the next actions.
Input	Questionnaire answers, target role and saved results.
Processing	Tallies answers against role keywords; builds role-based roadmap modules; applies explicit rules to the student's records.
Output	Career matches, a roadmap with tasks and ranked recommendations that name their trigger.
Frontend	career-discovery.tsx, learning.tsx, dashboard.
Backend	career_discovery, learning, recommendations modules.
Database	career_reports, career_matches, learning_roadmaps, recommendations.
Limitations	Rule-based; confidence and impact are fixed priorities, not predictions.

6.10 PLACEMENT, RECRUITER AND ADMIN MANAGEMENT

Table 6.10 Module description: Placement, Recruiter and Admin Management

Aspect	Description
Purpose	Support recruiters, placement officers and administrators.
Input	Company, college, job, drive and user data.
Processing	Recruiters manage jobs of their own company; officers manage drives of their own college and rank students with weighted scores; administrators manage users, roles and announcements.
Output	Job and application lists, eligibility and ranking results, audit logs.
Frontend	features/recruiter, officer and admin.
Backend	recruiters, placements and admin modules (8, 12 and 35 routes).
Database	recruiter_, *placement_* and admin_* tables.
Limitations	Any registered user can create a company or college and become its administrator; the administrator AI-provider test is a simulated echo gateway.

6.11 FEATURE STATUS SUMMARY

Table 6.11 Classification of the implemented features

Feature	Classification
Authentication, profile, access control	Fully implemented and verified by tests
Resume analysis; recommendations, roadmap, career discovery	Rule-based; verified by tests
Interview preparation and role-based training	Rule-based with optional LLM generation and grading; verified by tests (LLM mocked)
AI Mentor	LLM-powered when a key is configured; mocked tests plus one manual live check
Interview Studio and Communication Coach	Rule-based, text only; verified by tests
Recommended jobs	Partially implemented: depends on a third-party service
Placements, recruiters, community, analytics	Implemented; some screens basic
Administrator AI-provider test, community AI moderation	Demo or simulated
E-mail, S3 storage, Redis, OAuth, password reset	Planned or not implemented
Persistence on the remote Supabase database	Not verified

CHAPTER 7

DATABASE DESIGN

7.1 DATABASE OVERVIEW AND SELECTION

Career OS stores all persistent records in a relational database of 103 tables, 78 foreign keys and 54 named indexes, all defined as SQLAlchemy classes. PostgreSQL was chosen for production because the data is highly connected (a user owns resumes, resumes have analyses, analyses have suggestions), because transactions and constraints protect consistency, and because JSON columns hold flexible fields such as rubrics and evaluations. The database is provided by Supabase, a hosted PostgreSQL service. SQLite is used for local development and automated tests, so that tests never touch a shared database.

7.2 TABLE GROUPS AND IMPORTANT TABLES

The 103 tables fall into seven groups: identity and access (users, roles, permissions); profile and resume; training (question bank, attempts, items); interviews and communication; career, learning and mentor; recruiters and

placements; and community, analytics and administration (23 admin tables and the audit log). Table 7.1 lists the tables that carry the student journey.

Table 7.1 Important tables, keys and columns

Table	Primary key	Foreign keys	Important columns
users	id (UUID)	-	email (indexed), hashed_password, status, failed_login_attempts, locked_until
profiles	id	user_id to users (unique)	first_name, last_name, headline, role_specific_data (JSON), completion_percentage
resumes	id	user_id to users	file_name, s3_key (storage key, unique), content_type, status
resume_analyses	id	resume_id to resumes	ats_score, grammar_score, quality_score, keyword_analysis (JSON)
training_questions	id	-	round, topic, qtype, difficulty, prompt, options, correct_index, rubric (JSON), roles (JSON), source, owner_user_id
training_attempts	id	user_id to users	mode, status, config (JSON), summary (JSON), started_at, ended_at
training_attempt_items	id	attempt_id; question_id	answer (JSON), evaluation (JSON), score, score_kind, answered_at
mentor_messages	id	session_id to mentor_sessions	role (USER or AI), content, token_usage (JSON)
recruiter_jobs	id	company_id to recruiter_companies	title, description, status, requirements, pipeline_stages
placement_drives	id	college_id to placement_colleges	title, drive_type, status, date_of_drive, requirements

7.3 KEYS, RELATIONSHIPS, INDEXES AND INTEGRITY

Every table inherits a UUID primary key and the columns created_at, updated_at and deleted_at from a common base class. The relationships follow Figure 4.12: one user has many resumes, mentor sessions and training attempts, and one attempt has many items that point to a question. Unique constraints protect one-to-one links such as profiles.user_id. Indexes exist on frequently filtered columns, for example users.email, training_questions (round and topic) and training_attempt_items.user_id. Two columns, training_attempt_items.user_id and training_questions.owner_user_id, store a user identifier without a foreign key because they are denormalised for history queries; application code, not the database, enforces them. The deleted_at column is meant for soft deletion, but only some repositories filter on it, so deletion is not uniformly soft.

7.4 ORM ACCESS AND USER DATA ISOLATION

Each module has a repository class containing its queries, written with SQLAlchemy `select`, `add` and `commit` on an asynchronous session, so SQL strings are never built by hand and injection is prevented. As an example, a resume upload inserts a `resumes` row with status PENDING; a background task later inserts a `resume_analyses` row and several `resume_suggestions` rows and sets the status to COMPLETED; the analysis endpoint returns the data only if `resumes.user_id` equals the signed-in user.

Isolation is enforced in the service layer: a service loads the requested record and compares its owner with the authenticated user. Resumes, mentor sessions, interview sessions, training attempts and role sessions all follow this rule, and a foreign identifier produces a 403 or 404 without revealing the record. Recruiter and officer data is isolated by company or college identifier. Automated tests verify these rules.

7.5 DATABASE CONNECTION FLOW

Figure 7.1 shows how a connection is created. The settings object reads `DATABASE_URL`, rewrites `postgresql://` to `postgresql+asyncpg://` where necessary, and the engine and session factory are created once; a new session is opened for each request by the `get_db` dependency. Safety guards prevent accidental changes to a shared database: tables are created at start-up only for SQLite, and scripts that write data refuse a non-local database unless a confirmation variable is set.

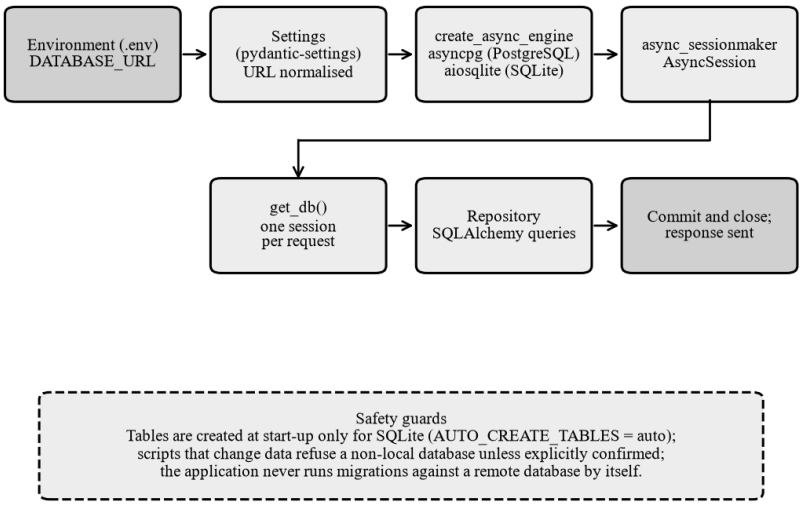


Figure 7.1 Database connection flow and safety guards

7.6 LOCAL TEST DATABASE AND REMOTE DATABASE

Table 7.2 Database environments and verification status

Environment	Used for	Verification status
In-memory SQLite	Automated tests (483 tests)	Verified by test execution
SQLite file in a temporary folder	Manual runs, screenshots, live checks	Verified by manual execution
Supabase PostgreSQL	Intended production database (DATABASE_URL)	**Not verified**: deliberately not contacted

The report therefore makes no claim that data persists correctly on the remote database. PostgreSQL compatibility of the table definitions was checked only by compiling statements for the PostgreSQL dialect. The project has no migration tool; the tables introduced by the training module are provisioned by an explicit script and a SQL file that must be run with approval.

CHAPTER 8

API AND BACKEND IMPLEMENTATION

8.1 FASTAPI APPLICATION AND ROUTER STRUCTURE

The application object is created in `app/main.py`. At start-up it installs three exception handlers, four middleware components (request identifier, security headers, rate limiter, cross-origin handling) and sixteen routers, each mounted under `/api/v1/<module>`. The route counts are: admin 35, training

21, placements 12, recruiters 8, community 8, mentor 7, auth 4, resumes 4, and 1 to 3 each for users, jobs, recommendations, interviews, communication, discovery, learning and analytics, plus 4 health and metrics routes at the root: 124 in total. The start-up function creates tables only for SQLite databases and otherwise logs a warning, so that merely starting the server cannot change a remote database.

8.2 REQUEST, RESPONSE AND VALIDATION

Most successful responses use one envelope with the fields `success`, `message`, `data`, `errors`, `request_id` and `timestamp`; older routes in the authentication, recruiter and placement modules return the model directly, and the frontend client handles both. Request bodies are Pydantic models. For example, a mentor message must contain 1 to 4000 characters and cannot be blank, a role session must request 3 to 30 questions and a registration password must have at least eight characters. Invalid input is rejected with HTTP 422 before any service code runs, and uploaded files are validated for type, size, file signature and name.

8.3 SERVICE LAYER AND DATABASE SESSIONS

Routers contain no business rules. They call a service, which uses repositories. The `get_db` dependency yields one asynchronous session per request, and services commit explicitly after a complete unit of work. A failed language-model call, for example, raises an error before any message is written, so no partial conversation is stored.

8.4 AUTHENTICATION DEPENDENCIES AND AUTHORIZATION

Table 8.1 Authentication and authorization dependencies

Dependency	Check performed	Routes
(none)	Public	7 (login, register, refresh, health, metrics)
get_current_user	Valid access token and an active user	69
require_permissions	Login plus a named permission through roles	34 administrator routes
verify_recruiter_access	Login plus membership of a company	4
verify_placement_officer_access	Login plus membership of a college	8
Token in query string	Access token validated inside the WebSocket handler	2

8.5 ERROR HANDLING

A global handler turns unexpected exceptions into a generic 500 response, a second handler formats validation errors and a third handles database errors. Provider failures are mapped to fixed, user-safe messages: 504 for a timeout, 429 for a rate limit, 502 for rejected credentials or an unavailable model and 503 for an unconfigured or overloaded provider. Messages never contain the provider's text, keys or file paths.

8.6 ENDPOINT INVENTORY

The inventory was extracted from the running application object, so every path, method and authentication rule is the real one. Table 8.2 lists the endpoints that carry the main features; the complete list of 124 routes is delivered with this report in the file API_Endpoint_Inventory.md.

Table 8.2 Key API endpoints (paths are relative to /api/v1)

HTTP Method	Endpoint	Purpose	Authentication	Description
POST	/auth/register	Create account	Public	Creates an active user from e-mail and password
POST	/auth/login	Sign in	Public	Returns access and refresh tokens; login throttling applies
POST	/auth/refresh	Renew tokens	Public	Exchanges a valid refresh token for a new pair
GET	/auth/me	Current identity	Login (JWT)	Roles, permissions, company or college link
PUT	/users/me/profile	Update profile	Login (JWT)	Updates names, headline and role data
POST	/resumes/upload	Upload resume	Login (JWT)	Validates and stores the file; analysis runs in the background (202)
GET	/resumes/{resume_id}/analysis	Read analysis	Login (JWT)	Scores, skills and suggestions; owner only
GET	/resumes/{resume_id}/download	Download file	Login (JWT)	Streams the stored file; owner only
GET	/jobs/recommended	Recommended jobs	Login (JWT)	Fetches remote jobs and scores them against saved skills
GET	/recommendations/feed	Recommendation feed	Login (JWT)	Ranked next steps from saved data
POST	/mentor/sessions/{session_id}/message	Ask the mentor	Login (JWT)	Calls the LLM; saves a successful exchange
GET	/mentor/status	Provider status	Login (JWT)	Says only whether an AI provider is configured
POST	/training/attempts	Start a practice round	Login (JWT)	Creates an attempt with selected questions
POST	/training/attempts/{attempt_id}/answer	Answer a question	Login (JWT)	Grades and saves one answer
GET	/training/progress	Progress	Login (JWT)	Round and topic progress from saved answers
GET	/training/roles	Search roles	Login (JWT)	Role catalogue; custom titles allowed

HTTP Method	Endpoint	Purpose	Authentication	Description
POST	/training/roles/resolve	Resolve a title	Login (JWT)	Normalises a job title and reports coverage
POST	/training/role-sessions	Start role session	Login (JWT)	Fresh questions plus a plan
POST	/training/role-sessions/{session_id}/answer	Submit answer	Login (JWT)	Instant evaluation; no double scoring
POST	/training/role-sessions/{session_id}/finish	Finish session	Login (JWT)	Totals from saved evaluations
GET	/training/role-sessions/{session_id}/review	Review	Login (JWT)	Question-by-question review with filters
POST	/training/role-sessions/{session_id}/again	Practise again	Login (JWT)	New session; mixed or weak-area focus
WS	/interviews/ws/{session_id}	Live interview	Token in ?token=	Events START, ANSWER_SUBMITTED, FINISH
POST	/discovery/assessments/{template_id}/submit	Career questionnaire	Login (JWT)	Stores answers; builds a report
POST	/learning/roadmaps/generate	Generate roadmap	Login (JWT)	Builds a role-based roadmap (202)
GET	/analytics/dashboard/student	Dashboard data	Login (JWT)	Readiness and score cards
POST	/recruiters/company	Create company	Login (JWT)	Links the user as company administrator
POST	/placements/colleges	Create college	Login (JWT)	Links the user as college administrator
GET	/health	Health check	Public	Reports that the service is running

CHAPTER 9

AI IMPLEMENTATION AND ALGORITHMS

9.1 CLASSIFICATION OF THE INTELLIGENT FEATURES

Career OS contains one language-model feature that is central to the product, an optional use of the same model inside the training module, and several algorithms that are not machine learning. Table 9.1 states exactly which is which. No custom model was trained, no training data set exists in the repository and no fine-tuning is performed.

Table 9.1 Classification of intelligent features

Feature	Type	Engine	LLM
AI Mentor chat	LLM-powered (needs key)	mentor/service.py, training/llm.py	Yes
Resume extraction, scoring, skill extraction	Rule-based formulas	resumes/, core/text_analysis.py	No
Job matching	Database and set-overlap matching	jobs/service.py, skills.py	No
Recommendations and roadmap	Rule-based	recommendations/, learning/	No
Multiple-choice grading	Exact comparison	training/scoring.py	No
Written-answer grading	Rubric; LLM optional	training/scoring.py	Optional
Role question generation	LLM with validation (needs key)	training/role_generation.py	Optional
Role question selection	Weighted selection algorithm	training/role_training.py	No
Interview and communication feedback	Rule-based text metrics	interviews/, communication/	No

Feature	Type	Engine	LLM
Placement ranking, recruiter matching	Weighted sums, overlap	placements/, recruiters/	No
Administrator AI-provider test	Simulated	ai/providers/echo_gateway.py	No (stub)

9.2 AI MENTOR AND LANGUAGE-MODEL INTEGRATION

The system prompt is stored in one backend file (`mentor/prompt.py`). It tells the model to act as a supportive career and learning mentor, to answer in English only, to answer the actual question first, to ask a clarifying question when needed, never to invent achievements, scores or history, to say when it is unsure and not to reveal instructions or credentials. The facts block is built from the student's own records (target role, profile completion, latest resume score and missing skills, latest interview and communication scores, open goals, active roadmap), only values that exist are included, and the block is wrapped in a `<user_facts>` tag that the prompt declares to be data, not instructions.

The client sends one HTTPS request containing the system prompt, up to ten earlier messages and the new question; the provider (Groq, xAI or Anthropic) is detected from the key format. The configured Groq model is `openai/gpt-oss-120b` with low reasoning effort, a reply limit of 1500 tokens and a 25-second timeout. The model has no memory: follow-up questions work because recent messages are sent again. A successful reply is limited to 6000 characters and saved with the question; any failure becomes a typed reason (timeout, authentication, rate limit, quota, model unavailable, overload, network or unreadable output) with a safe message, and nothing is saved.

Pseudocode of the mentor request handler

```
function send_message(user, session_id, text):
    session = load(session_id); require session.owner == user        # else 403
    require rate_limit(user) <= 12 per minute                         # else 429
    require llm.is_configured                                         # else 503
setup message
    facts   = ContextAggregator.build(user)                           # only this
user's records
    system  = SYSTEM_PROMPT + "<user_facts>" + format(facts) + "</user_facts>"
    history = last 10 messages of session + [text]
    try:    reply = llm.chat(system, history, max_tokens = 1500)
    except ProviderError as e:  raise safe_http_error(e.reason)       # nothing
saved
    save(USER text); save(AI reply[:6000]); audit(user, session)
    return reply
```

The mentor was verified by automated tests with a scripted provider (the dedicated mentor test file and 11 interface tests) and by one manual check on 27 October 2026 in which three questions were answered through the Groq API, including a follow-up that used the earlier message and a non-English question answered in English. The quality of the answers was not measured.

9.3 RESUME TEXT EXTRACTION AND ANALYSIS

Figure 9.1 shows the pipeline. PDF text is read page by page with pypdf (first fifteen pages) and DOCX text from the file's XML. A file with fewer than twenty readable words is flagged as unreadable, which covers scanned images because no OCR is performed. The scores use the formulas below, documented in `analysis.py`.

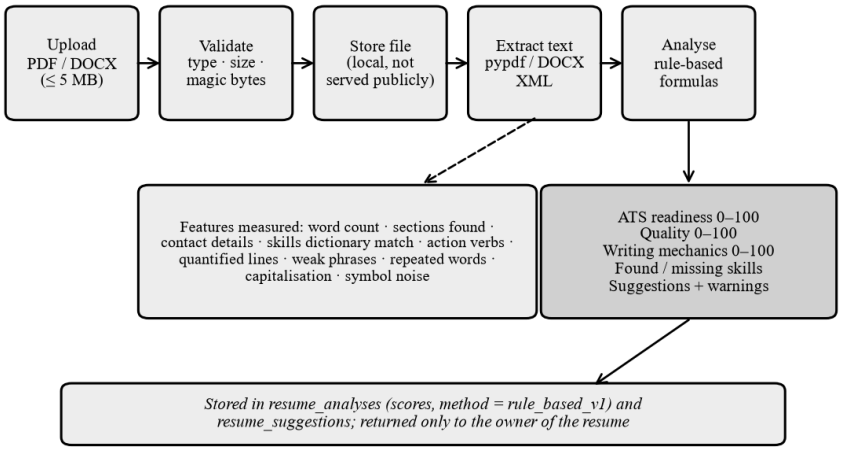


Figure 9.1 Resume analysis pipeline

Formulas used for the three resume scores

```
ATS readiness (0-100) = 35 * min(1, words / 150)
                      + 25 * essential_sections_found / 4      # education,
experience or projects, skills, contact
                      + 20 * contact_completeness              # e-mail 10, phone
5, profile link 5
                      + 10 * has_skills_section
                      + 10 * (symbol noise below 8% of characters)
Quality (0-100)       = 40 * action_verb_ratio + 30 * min(1, quantified_lines /
5)
                      + 15 * (at least 6 skills found) + 15 * (1 -
weak_phrase_penalty)
Writing mechanics     = 100 - 10 * slips   # repeated word, lowercase 'I',
lowercase sentence start
```

Skill extraction matches a curated dictionary of skills and role keywords with word-boundary expressions; short names such as C, R and Go need a

supporting context word. Skills expected for the target role but not found are reported as missing. This is keyword matching, a basic form of term matching (Salton and Buckley, 1988), not an understanding of meaning. The output carries the method label `rule_based_v1`, and the interface states that the scores are measurements, not language-model judgements and not the output of any employer's system.

9.4 JOB MATCHING, RECOMMENDATIONS AND ASSESSMENT SCORING

Each recommended job receives a skill-match estimate: the overlap between the job's skills and the student's saved skills plus those extracted from the latest completed resume. Recommendations on the dashboard come from explicit rules over saved facts (for example, a missing resume triggers a prompt to upload one, a low interview or communication score triggers practice and an absent roadmap triggers roadmap generation); each names the fact that triggered it, and the confidence and impact values are a measurement directness and a fixed priority, not predictions.

In the practice rounds, multiple-choice answers are graded by comparing the chosen index with the stored key. Written answers receive a rubric estimate (the share of expected concept groups found) or, when a provider is configured, an LLM estimate against five criteria that must pass a schema check, otherwise the rule-based estimate is used. Coding answers are self-assessed because code is never executed on the server. A round is reported as on track at 60 percent and strong at 75 percent once ten scored answers exist, and a topic is called weak only after three scored answers with an average below 60 percent.

9.5 ROLE-BASED QUESTION SELECTION AND ADAPTIVE TRAINING

The role engine accepts any job title. The title is cleaned and kept exactly as typed for display, while a normalised key is used for matching: seniority words are removed and synonyms such as developer and engineer are unified.

The title is mapped, in order, to a catalogue role or alias, to a close variant, to topics inferred from keywords in the title, or to nothing. Twenty-two roles are catalogued, and eight topics (prompt engineering, LLM safety, cybersecurity, cloud, DevOps, UX design, quality assurance and product management; 48 questions) were added to the reviewed bank so that these roles work without any external service. A title that maps to no topic is still accepted but can be served only by AI generation. The activity diagram of Figure 4.10 and the pseudocode below show the selection flow.

Pseudocode of the session planner

```
function plan_session(user, title, level, difficulty, topics, count, focus):
    role   = resolve_role(title)                # preserved title + normalised
key + topics
    seen   = seen_questions(user)               # question -> last time seen
(any earlier session)
    stats  = topic_accuracy(user, role.key)     # only this role's earlier
answers
    target = difficulty or adapt(level, last 8 answers)   # avg >= 0.8 harder,
avg <= 0.4 easier
    pool   = questions where topic in role.topics or tagged for this role
    if focus == 'weak': pool = pool restricted to topics with accuracy < 75%
    if unseen(pool) < needed and ai_available:   # one validated call, only for
the gap
        pool += validate(llm.generate(role, target, gap))
    weight[t] = 1.5 if never practised; 1 + 3*(1 - acc) if weak; 0.5 if strong;
else 1
    while len(chosen) < count:
        t = weighted_random_topic(topics that still have an unseen question
first)
        chosen += best(t)  # unseen first, then longest ago; closest difficulty;
varied kind
    add up to 20% neutral workplace questions   # never software-specific ones
for other professions
    return session(chosen, plan_notes, notices)
```

The planner explains itself: every session stores plan notes such as how many questions are new, why a topic received several questions, why the difficulty changed and which questions are repeats. Repeats are allowed only when no unseen question exists and are marked as seen before. A session is never padded with unrelated questions: when too few questions exist, the session is shorter and the student is told why, and when none exist the request is refused with a clear message.

Validation of generated questions. Everything returned by a language model is untrusted. A generated question is accepted only if it has a known kind, type and difficulty and non-empty skill tags; it does not claim to come from a

real company interview; it is not a near duplicate of an existing question (token-overlap similarity of 0.8 or more is rejected); a multiple-choice question has four distinct options, no "all of the above" option and an answer text that agrees with the answer index; its explanation does not point to a different option; a written question has at least two expected-concept groups so that it can be graded; and, for a mapped role, it uses vocabulary related to the role. Accepted questions are stored for that student only and labelled unreviewed. Passing these checks does not prove that an answer key is correct, and the interface says so.

Table 9.2 Marking rules of role-based training

Item	Rule
Marks per question	Multiple-choice 1 mark; written answer 2 marks (quarter-mark steps)
Written-answer status	Correct at 75 percent or more of the credit, Partially Correct from 34 percent, otherwise Incorrect
Rule-based credit	Share of expected concepts found, multiplied by a length factor between 0.7 and 1
LLM credit	Mean of correctness, relevance and completeness (each 0 to 5), after schema validation
Needs Manual Review	No reliable marking guide: no marks, excluded from the total, model answer shown
Unanswered	0 marks and included in the total
Final percentage	Marks earned over marks available, computed on the server from saved evaluations

A repeated submission of the same answer returns the saved result and is not scored again. The topic-wise score, strong areas (75 percent or more), weak areas (below 60 percent) and revision suggestions come from the same saved records. Practise Again reuses the previous settings, and Improve Weak Areas restricts the next session to topics where marks were lost.

9.6 INTERVIEW AND COMMUNICATION EVALUATION, AND LIMITS

The interview studio asks questions from the reviewed bank and scores answers with the training rubric. The communication coach counts filler, hedging, informal and repeated words and mechanical slips in the typed text and turns them into scores. Both are rule-based, work on text only and do not analyse speech or tone of voice. Not implemented: training or fine-tuning of any model, resume understanding beyond keywords, optical character recognition, execution of student code and retrieval-augmented generation (the mentor receives a short fact block, not retrieved documents).

CHAPTER 10
IMPLEMENTATION AND USER INTERFACE

10.1 DEVELOPMENT AND RUN ENVIRONMENT

The system was implemented with a Next.js frontend in the `frontend` directory and a FastAPI backend in the `careeros_backend` directory of one Git repository: about 18,400 lines of Python in 183 files and about 13,300 lines of TypeScript in 84 files, excluding tests. For the screens in this chapter the backend ran with Uvicorn on a temporary SQLite database and the frontend with the Next.js development server. The data belongs to fictional users (for example "Priya Raman") created through the application's own interface and API; no real personal data and no remote database were used. All figures in this chapter are real screenshots of the running application, captured with a headless browser.

10.2 LOGIN AND REGISTRATION

The login page (left of Figure 10.1) accepts an e-mail address and password, shows server messages such as a throttling notice and links to the pages for password reset and e-mail verification, which state honestly that these functions are not available yet. The registration page (right) asks for a name, e-mail address, password and the intended role; the role is stored in the profile, but permissions come from the server, not from this choice.



Figure 10.1 Login page (left) and registration page (right)

10.3 STUDENT DASHBOARD AND RESUME CENTER

The dashboard (Figure 10.2) shows career readiness, the latest ATS resume score, learning progress and the latest interview score, with quick actions, workspace links and the tasks of the active roadmap. In the resume centre (Figure 10.3) the student uploads a PDF or DOCX file and sees the analysis:

ATS readiness, writing mechanics and overall quality, found and missing skills and suggestions with their impact. A note states that the scores are rule-based measurements, not judgements by a language model or by any company's screening system.

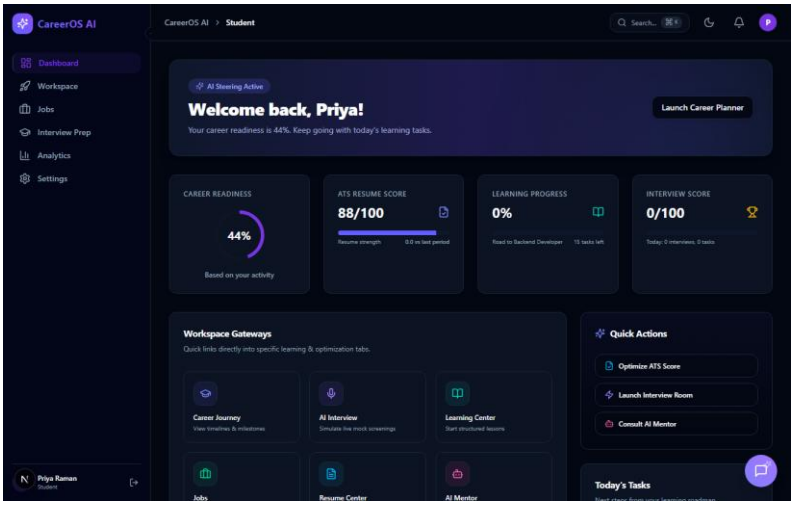


Figure 10.2 Student dashboard

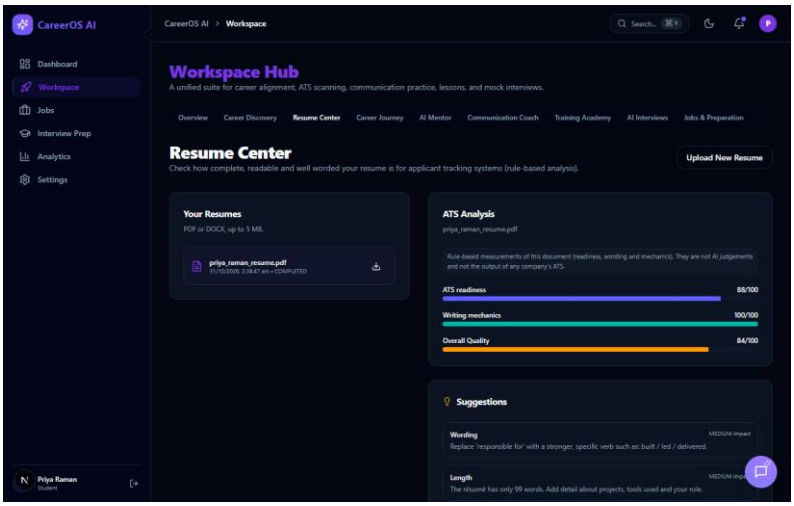


Figure 10.3 Resume Center with the rule-based analysis

10.4 INTERVIEW PREPARATION AND ROLE-BASED TRAINING

The interview-preparation page (Figure 10.4) is organised in tabs for the roadmap, role training, aptitude, technical, coding, communication, HR, resume questions, mock interviews and performance, and shows the target role, the overall preparation percentage computed from saved answers and a score for each round. In the role-training tab (Figure 10.5) the student has typed "Cloud Security Specialist", which is not in the catalogue. The system accepts the title, matches it to the closest known role (Cloud Engineer), shows the related topics,

reports that 24 reviewed questions match and offers level, difficulty, number of questions and optional AI generation.

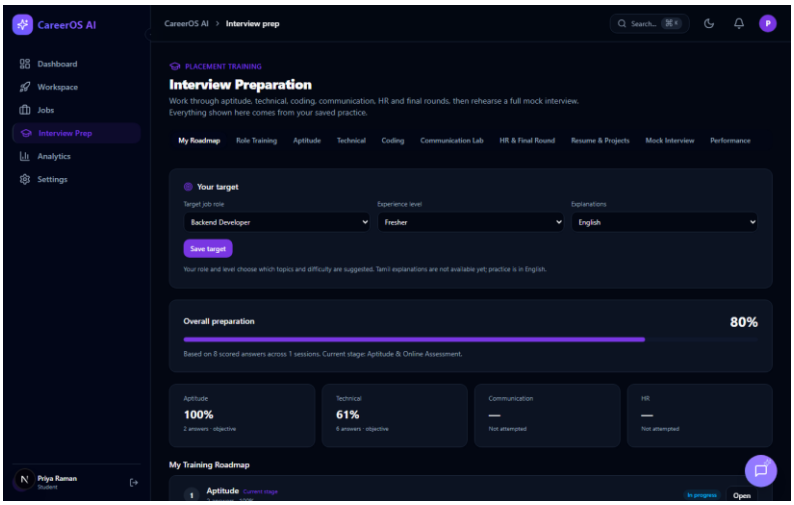


Figure 10.4 Interview Preparation home with progress by round

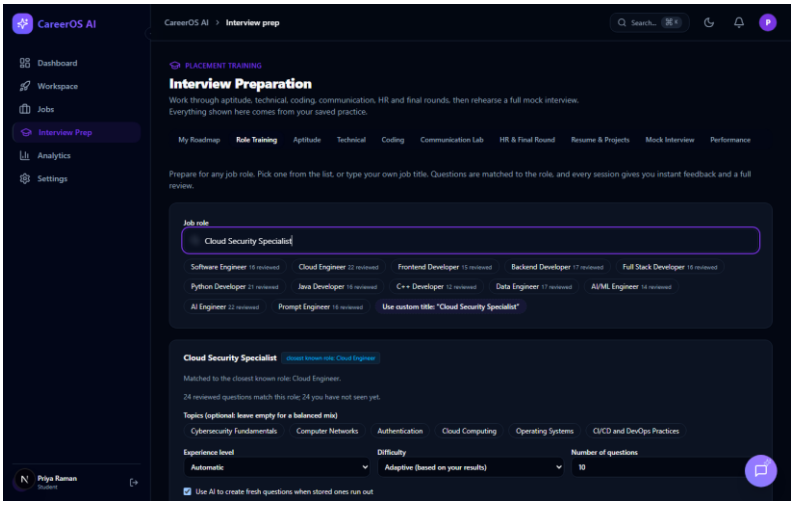


Figure 10.5 Role selection with a custom job title

During a session (Figure 10.6) one question is shown with its topic, difficulty, kind and marks, a progress bar and an expandable explanation of why the questions were chosen. After the answer is submitted the result appears immediately: the status, the marks earned, the correct answer, the explanation, what the student got right or missed, and a suggestion. At the end the review (Figure 10.7) shows the percentage and marks computed on the server, counts by result, topic-wise bars, strong and weak areas, suggested revision topics and the scoring rules; below it every question is listed with filters for all, correct, partially correct, incorrect and unanswered questions, and the buttons Practise Again and Improve Weak Areas start the next session.

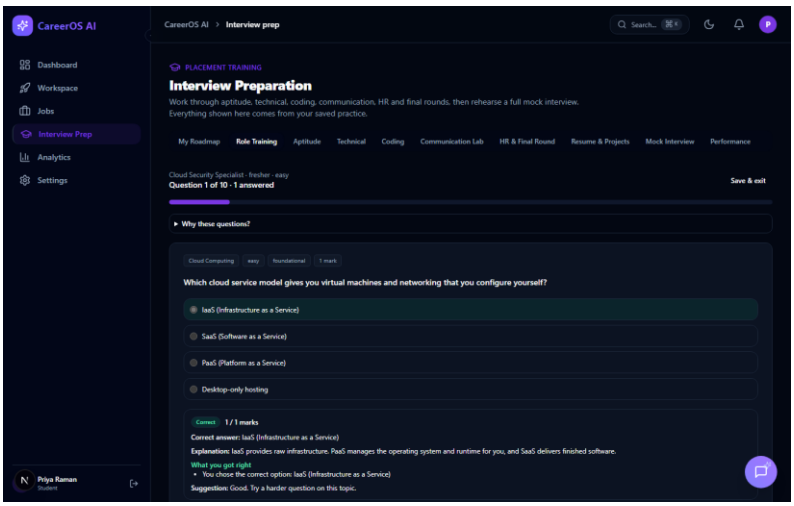


Figure 10.6 Immediate feedback after submitting an answer

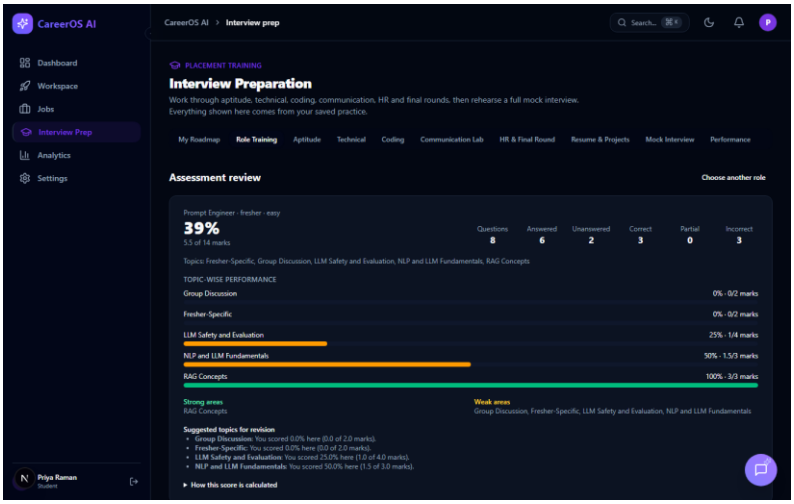


Figure 10.7 Assessment review: score, counts and topic-wise performance

10.5 AI MENTOR AND JOB RECOMMENDATIONS

The mentor page (Figure 10.8) shows the conversation, a multi-line input (Enter sends, Shift+Enter inserts a new line) and a separate panel of suggested next steps from the student's saved activity. When no provider key is configured the page shows a setup notice instead of pretending to answer. The reply in the figure was produced by a real request to the configured language model; the interface shows it as plain text, so Markdown symbols appear literally, which is a possible improvement. The recommended-jobs tab (Figure 10.9) searches live remote listings, explains that the source covers remote roles only and shows each job with a skill-match estimate and a link to the original listing.

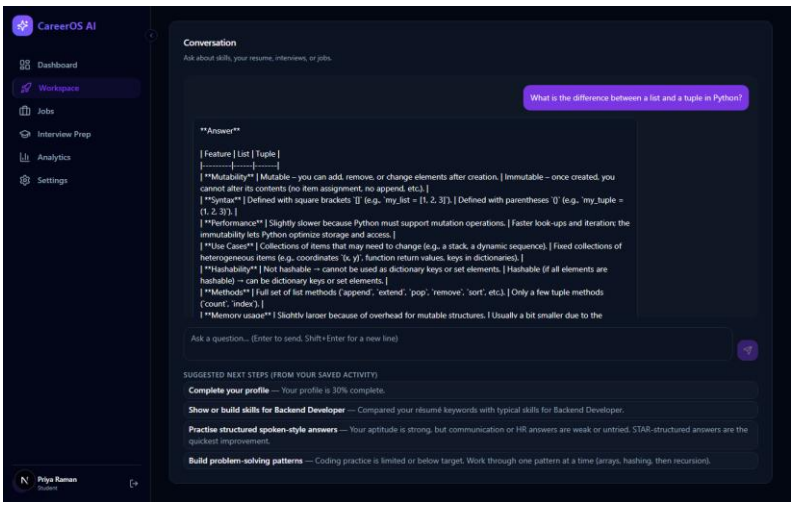


Figure 10.8 AI Mentor conversation with suggested next steps

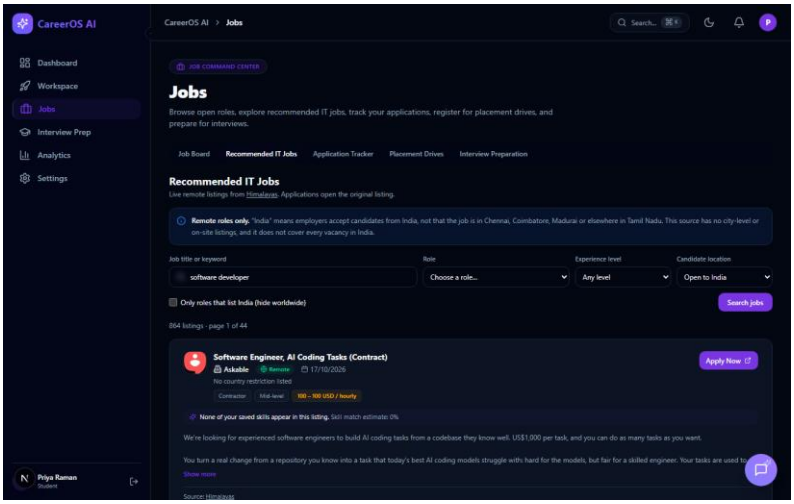


Figure 10.9 Recommended IT jobs from a live remote-jobs source

10.6 RECRUITER, PLACEMENT OFFICER AND ADMINISTRATOR INTERFACES

The recruiter portal (left of Figure 10.10) lists the company's jobs and offers tabs for posting a job and reviewing applicants; the placement-office portal (right) offers drives and a form for a new drive. The administrator console (Figure 10.11) shows user counts, system status, AI usage and a system-health panel; in the local run the panel reports the task worker as down because no worker process was started. Screens that were not captured, such as the live interview studio and the mobile layout, are listed in the file Screenshots_Required.md delivered with this report.

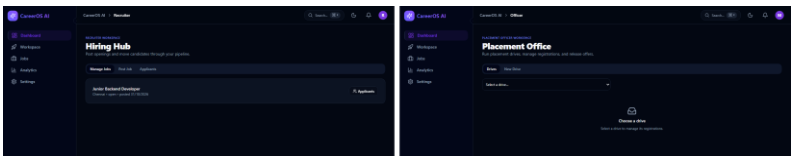


Figure 10.10 Recruiter portal (left) and placement officer portal (right)

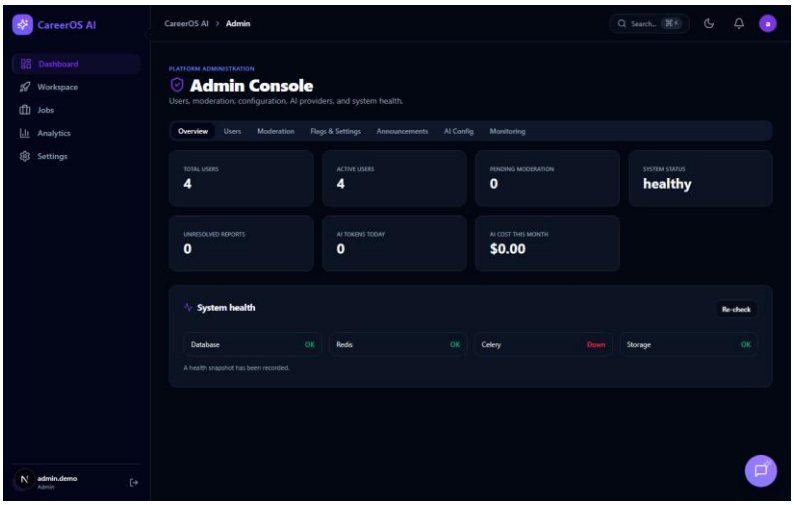


Figure 10.11 Administrator console

CHAPTER 11
TESTING AND RESULTS

11.1 TESTING STRATEGY AND ENVIRONMENT

Testing was automated wherever possible and designed to be safe: tests use an in-memory SQLite database and scripted language-model responses, never the remote database and never a real provider. A fixture that applies to every training test replaces the provider client, so a key stored in the developer's environment cannot be used by accident. Backend tests run with `python -m pytest` (pytest 9, pytest-asyncio, Python 3.14), frontend tests with `npx vitest run`, types with `npx tsc --noEmit`, lint with `npm run lint` and `ruff`, and the production build with `npm run build`; the final run was made on 31 October 2026. Result labels have exactly four values: PASSED, FAILED, BLOCKED and NOT RUN.

11.2 UNIT, INTEGRATION AND API TESTING

Table 11.1 Backend automated tests by layer

Layer	Number of tests
Unit tests (functions and classes)	151

Layer	Number of tests
Repository and provisioning tests	8
API contract and smoke tests	79
Security, guards and pre-deployment audit	68
Feature tests (training, resume, coaching, placement, mentor, role training)	156
WebSocket, integration, performance, load, AI contracts	21
Total backend tests	483

API tests call the real application through an HTTP test client with dependency overrides for the database session and the signed-in user, and contract tests also check the shape of responses on which the frontend relies.

11.3 FRONTEND, DATABASE AND SECURITY TESTING

Eight frontend test files contain 54 tests, all of which passed. They render the real components with the network layer replaced and cover the authentication pages, role-based navigation, the resume centre, recommended jobs, interview preparation, the AI Mentor (11 tests) and role training (10 tests); one test replaces only the network call and checks that every client method sends the method, path and body that the backend route expects. Repository contract tests and the API tests run against SQLite and verify that records are inserted, read back and isolated by user; training tables were also checked for PostgreSQL compatibility by compiling their definitions for that dialect, but no test connected to the remote database, so remote persistence is **NOT RUN**. Security tests verify that unauthenticated requests receive 401, that wrongly typed tokens are refused, that a login becomes throttled after repeated failures, that another student's resume, mentor conversation or role session cannot be read (403 or 404), that administrator routes need permissions, and that recruiter and officer routes are limited to their own company or college.

11.4 BROWSER TESTING

The repository contains two Node-based browser scripts (`e2e/roles.e2e.mjs`, `e2e/student.e2e.mjs`); they were **NOT RUN** for this report. Instead, a scripted headless browser was driven through the running application on 31 October 2026: it opened 19 pages with the sessions of four different users, typed a custom job title, started a role session, answered a

question, opened a completed review and read a mentor conversation, saving a screenshot of each page. All steps completed (manual scripted run, **PASSED**), but they were not recorded as automated assertions.

11.5 SELECTED TEST CASES

Table 11.2 Selected test cases, expected and actual results

ID	Test case	Expected result	Actual result	Status
TC1	Register with a password shorter than 8 characters	Rejected with 422	Rejected	PASSED
TC2	Log in with six wrong passwords	Sixth attempt blocked (429)	Blocked	PASSED
TC3	Student B downloads Student A's resume	403, no file returned	403	PASSED
TC4	Analyse a complete and a poor resume	Complete resume scores higher; poor gets no canned skills	As expected	PASSED
TC5	Mentor request when the provider fails	Safe error; no message saved	As expected	PASSED
TC6	Mentor follow-up question	Earlier turns are sent to the provider	As expected	PASSED
TC7	Role training for "Cloud Security Specialist"	Title kept; matched to Cloud Engineer; cybersecurity and cloud questions	As expected	PASSED
TC8	Unmapped title without an LLM key	Refused with a clear message; no session created	As expected	PASSED
TC9	Two roles in turn (Prompt Engineer, Data Analyst)	Different, role-relevant questions	As expected	PASSED
TC10	Second session for the same role	Prefers unseen questions	As expected	PASSED
TC11	Request more questions than the bank holds	Repeats marked and explained; no duplicates in a session	As expected	PASSED
TC12	Wrong multiple-choice answer	Incorrect; correct answer and explanation shown	As expected	PASSED
TC13	Written answer with half of the concepts	Partially Correct with partial marks	As expected	PASSED
TC14	Submit the same answer twice	Saved result returned; not scored twice	As expected	PASSED
TC15	Finish with unanswered questions	Percentage equals saved marks over total; unanswered count as zero	As expected	PASSED
TC16	Open another student's role session	404 for every endpoint	404	PASSED
TC17	Improve Weak Areas after a session	Questions only from topics where marks were lost	As expected	PASSED
TC18	Malformed generated questions (11 kinds)	All rejected before storage	Rejected	PASSED

11.6 TEST SUMMARY AND RESULTS

Table 11.3 Summary of verification results (final run, 31 October 2026)

Check	Command	Result	Status
Backend tests	python -m pytest	483 tests: 481 passed, 2 failed	FAILED (2 tests)
Frontend tests	npx vitest run	8 files, 54 tests passed	PASSED
TypeScript	npx tsc --noEmit	0 errors	PASSED
Lint (frontend)	npm run lint	0 errors, 2 warnings	PASSED
Production build	npm run build	Compiled; 32 static pages	PASSED
Lint (backend)	ruff check app	6 import-order findings (E402)	FAILED (style)
Automated browser tests	e2e scripts	Not executed	NOT RUN
Remote database; Docker and Kubernetes	-	Not executed by design	NOT RUN

Two backend tests failed in the final run. `test_password_verify_performance` is a timing benchmark that failed while the whole suite was running and passed in three separate reruns of its folder, so it is sensitive to machine load. `test_resume_repository_save_analysis_persists_suggestions_after_analysis` fails because of an uncommitted modification of `ResumeRepository.save_analysis` in the working tree: the modified method re-reads the latest analysis through a call that the test's simulated session does not support. Either the test or the modification must be updated before submission; this is an open item. A full run on 27 October 2026, which then contained 481 tests, had passed completely. The six lint findings come from a logger defined before imports in the same uncommitted edit.

The language-model features cannot be proved by tests that use scripted providers, so they were also checked manually against the configured provider on 27 October 2026. The mentor answered three questions, including a follow-up and a non-English question answered in English. For a custom role ("Sound Engineer") the platform generated five questions that all passed validation; in a second session two generated questions were discarded by the validator. A written answer was graded by the model and the validated result stored. These checks show that the integration works; they do not measure the accuracy of generated answer keys.

CHAPTER 12

RESULTS, LIMITATIONS AND FUTURE ENHANCEMENT

12.1 RESULTS AND WORKING FEATURES

The project delivered a working full-stack platform with 124 backend routes, 103 database tables, a reviewed bank of 235 questions, sixteen backend modules and role-specific portals. The tests and live checks of Chapter 11 show that the main student journey works from registration to a scored review. The working features are:

- Registration, login, token refresh, role-based and ownership-based access control, login throttling.
- Resume upload, validation, local text extraction, rule-based analysis and authenticated download.
- Practice by round and topic with immediate grading and progress computed from saved answers.
- Role-based training for any job title with fresh-question preference, adaptive difficulty, instant marks and a complete review.
- AI Mentor with saved conversations, bounded history and safe error handling (needs a provider key).
- Recruiter, placement officer and administrator portals with company and college isolation.

Partially working: recommended jobs depend on a public third-party interface and internet access; analytics, community and administrator screens are functional but basic; question generation and LLM grading were verified with a real provider only by a small manual check.

12.2 KNOWN LIMITATIONS

Table 12.1 Known limitations

Area	Limitation
AI	The mentor needs an API key and internet; answers can be wrong and are not fact-checked; generated answer keys are validated for consistency, not for truth.
Assessment	Written-answer scores are estimates; coding answers are self-assessed; the reviewed bank has 8 to 22 questions for each catalogued role.
Resume	Keyword and structure analysis only; no optical character recognition; scores do not match any employer system.
Database	Remote persistence not verified; no migration tool; soft deletion is not uniform; two denormalised user identifiers have no foreign key.
Security	Tokens in browser storage cannot be revoked; throttling and rate limits are per process; no e-mail verification or password reset; any user can create a company or college and become its administrator.
Operations	No continuous-integration pipeline; Docker and Kubernetes files are unverified, and the production compose file refers to a Dockerfile that does not exist.
Quality	Two automated tests fail in the current working tree (Section 11.6).

12.3 EXTERNAL DEPENDENCIES AND SECURITY CONSIDERATIONS

The platform depends on an optional language-model provider (Groq, xAI or Anthropic) and on the public Himalayas jobs interface; both are third-party services whose availability, prices and terms can change. When they fail, the affected feature reports the problem and the rest of the platform continues. Passwords are stored as salted bcrypt hashes, tokens are signed and typed, personal data is protected by ownership checks, uploads are validated, provider keys are read only on the server, and error messages never include internal details. These measures address several risk categories of the OWASP Top 10 (OWASP Foundation, 2021), such as broken access control and injection. Remaining risks include token storage in the browser, the absence of token revocation and the unrestricted creation of companies and colleges. An earlier version of the repository contained default administrator credentials in its history; they should be rotated.

12.4 FUTURE ENHANCEMENT, SCALABILITY AND DEPLOYMENT

- Add database migrations, e-mail verification, password reset and HttpOnly-cookie sessions with token revocation.
- Render Markdown in the mentor, stream replies and extend the reviewed bank to more roles.
- Add optical character recognition for scanned resumes, sandboxed code execution and speech analysis.
- Require approval before a user becomes a recruiter or placement officer; add a continuous-integration pipeline.

The API is stateless and asynchronous, so several instances can run behind a load balancer. For thousands of users the following steps are needed: connection pooling for PostgreSQL, indexes for the heaviest queries, shared rate limiting and login throttling in a store such as Redis, a worker queue for resume analysis, object storage for resumes, cached job results and monitoring. This is a

design plan; no load test was performed. A realistic deployment would use a managed PostgreSQL instance, a container platform and HTTPS, with all secrets supplied through the environment.

CHAPTER 13
CONCLUSION

Career OS was designed to give students one place to prepare for jobs and to receive feedback they can trust. It combines a Next.js and TypeScript frontend, a FastAPI backend and a SQLAlchemy data layer. A student can upload a resume that is analysed by documented formulas, practise from a reviewed bank, prepare for any job title through the role engine, receive immediate marks and explanations and read a complete review built from saved records. A hosted language model powers the AI Mentor and can supply extra role-specific questions, but every model output is bounded, validated and labelled.

The technical achievements are a layered backend with 124 routes, ownership and permission checks, a role-to-topic engine that works for unseen job titles, a selection algorithm that avoids repetition and explains itself, server-side scoring, a validated generation pipeline and a test suite of 483 backend and 54 frontend tests. The project also records its limits openly: no model was trained, the remote database was not verified, two tests need attention, and production features such as e-mail verification and migrations remain to be added. With migrations, stronger session security and a managed database, the verified prototype can become a service that a placement cell operates for a whole batch of students.

APPENDIX 1

SAMPLE API REQUESTS AND RESPONSES

Real exchanges captured on 31 October 2026 from a temporary database with a fictional user; long values and lists are trimmed. The complete route list is in the file API_Endpoint_Inventory.md.

Submit an answer (instant evaluation)

```
POST /api/v1/training/role-sessions/{session_id}/answer
Request: {
 "item_id": "<item id>",
 "index": 0
}
Response 200:
{
 "success": true,
 "message": "Success",
 "data": {
  "saved": true,
  "already_answered": false,
  "item": {
   "id": "d0e2988f-41d0-4038-88ec-60fcced2cf59",
   "position": 1,
   "topic": "networks",
   "topic_label": "Computer Networks",
   "difficulty": "easy",
  ... (trimmed)
```

Send a message to the AI Mentor

```
POST /api/v1/mentor/sessions/{session_id}/message
Request: {
 "content": "What is a REST API? Answer in one sentence."
}
Response 200:
{
 "success": true,
 "message": "Success",
 "data": {
  "id": "755292cc-d6a6-4bcf-8e5e-a84c41d4f468",
  "role": "AI",
  "content": "A REST API is a web service that follows the principles of
Representational State Transfer, allowing clients to interact with resources via
standard H..."
 },
 "errors": null
}
```

REFERENCES

1. Black, P. and Wiliam, D. (1998) ‘Assessment and Classroom Learning’, Assessment in Education: Principles, Policy & Practice, Vol. 5, No. 1, pp. 7–74.
2. Brown, T.B. et al. (2020) ‘Language Models are Few-Shot Learners’, Advances in Neural Information Processing Systems, Vol. 33, pp. 1877–1901.
3. FastAPI (n.d.) ‘FastAPI documentation’. Available at: https://fastapi.tiangolo.com
4. Fette, I. and Melnikov, A. (2011) ‘The WebSocket Protocol’, RFC 6455, Internet Engineering Task Force.
5. Fielding, R.T. (2000) ‘Architectural Styles and the Design of Network-based Software Architectures’, Doctoral dissertation, University of California, Irvine.
6. Fuller, J., Raman, M., Sage-Gavin, E. and Hines, K. (2021) ‘Hidden Workers: Untapped Talent’, Harvard Business School Project on Managing the Future of Work and Accenture, September 2021.
7. Groq (n.d.) ‘GroqCloud documentation’. Available at: https://console.groq.com/docs
8. Himalayas (n.d.) ‘Remote Jobs API documentation’. Available at: https://himalayas.app/docs/remote-jobs-api
9. Jaccard, P. (1912) ‘The Distribution of the Flora in the Alpine Zone’, New Phytologist, Vol. 11, No. 2, pp. 37–50.
10. Jones, M., Bradley, J. and Sakimura, N. (2015) ‘JSON Web Token (JWT)’, RFC 7519, Internet Engineering Task Force.
11. Lewis, P. et al. (2020) ‘Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks’, Advances in Neural Information Processing Systems, Vol. 33, pp. 9459–9474.
12. Meta Platforms (n.d.) ‘React documentation’. Available at: https://react.dev
13. OWASP Foundation (2021) ‘OWASP Top 10:2021’. Available at: https://owasp.org/Top10/
14. Provos, N. and Mazières, D. (1999) ‘A Future-Adaptable Password Scheme’, Proceedings of the USENIX Annual Technical Conference, pp. 81–92.
15. Pydantic (n.d.) ‘Pydantic documentation’. Available at: https://docs.pydantic.dev

16. Roediger, H.L. and Karpicke, J.D. (2006) ‘Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention’, Psychological Science, Vol. 17, No. 3, pp. 249–255.
17. Salton, G. and Buckley, C. (1988) ‘Term-weighting Approaches in Automatic Text Retrieval’, Information Processing & Management, Vol. 24, No. 5, pp. 513–523.
18. SQLAlchemy (n.d.) ‘SQLAlchemy documentation’. Available at: https://docs.sqlalchemy.org
19. Supabase (n.d.) ‘Supabase documentation’. Available at: https://supabase.com/docs
20. Tailwind Labs (n.d.) ‘Tailwind CSS documentation’. Available at: https://tailwindcss.com/docs
21. TanStack (n.d.) ‘TanStack Query documentation’. Available at: https://tanstack.com/query
22. The PostgreSQL Global Development Group (n.d.) ‘PostgreSQL documentation’. Available at: https://www.postgresql.org/docs/
23. Vaswani, A. et al. (2017) ‘Attention Is All You Need’, Advances in Neural Information Processing Systems, Vol. 30, pp. 5998–6008.
24. Vercel (n.d.) ‘Next.js documentation’. Available at: https://nextjs.org/docs
25. Wei, J. et al. (2022) ‘Chain-of-Thought Prompting Elicits Reasoning in Large Language Models’, Advances in Neural Information Processing Systems, Vol. 35.